

SAINT MARY'S
UNIVERSITY SINCE 1802

One University. One World. Yours.

PROJECT REPORT
MCDA5585 and MCDA5586

Submitted By:

Ms. Neeyati Mehta
A00477369

under the guidance of:

Dr. Pawan Lingras
Mr. Rob Moore

in partial fulfillment for the awarding of the degree:

Master of Science in
Computing and Data Analytics
November 2024

ACKNOWLEDGEMENT

I would like to express my heartfelt appreciation to everyone who contributed to the successful completion of this project report. Their guidance, support, and encouragement were pivotal to its success.

I am especially grateful to Dr. Pawan Lingras, the Director of the Master of Science in Computing and Data Analytics (MCDA) program at Saint Mary's University, for providing me with this invaluable opportunity. His constant support and guidance were crucial throughout the duration of this project.

I extend my deepest gratitude to Mr. Rob Moore, Chief Technical Officer at Wellnify. His technical expertise and insightful suggestions greatly enhanced the quality of this work. I am truly thankful for his guidance, which allowed me to explore new ideas and gain extensive knowledge.

I would also like to express my sincere thanks to Mr. Brendon Ferguson, Co-founder and COO of Wellnify. His vision and understanding of user perspectives in the technical realm have been incredibly enlightening and have broadened my view of the industry's future.

Special thanks to my project team members, whose collaboration and teamwork were essential to this project's success. Their knowledge, support, and constructive feedback were greatly appreciated.

Finally, I would like to thank both Saint Mary's University and Wellnify for giving me this incredible opportunity to gain industry exposure and work on such a sophisticated real-world project. This experience has been invaluable to my professional development.

CERTIFICATE

This is to certify that **Ms. Neeyati Mehta (ID: A00477369)**, a student at Saint Mary's University in the Master of Science in Computing and Data Analytics program, has completed the projects in fulfillment of the course requirements.

.....
Dr. Pawan Lingras

ACADEMIC SUPERVISOR

DEPARTMENT OF MATHEMATICS

AND COMPUTING SCIENCE

SAINT MARY'S UNIVERSITY

923 Robie St. Halifax, NS, Canada B3H 3C3

.....
Mr. Rob Moore

CHIEF TECHNICAL OFFICER

WELLIFY.AI

Halifax, NS, Canada B3J 1M5

TABLE OF CONTENTS

ACKNOWLEDGEMENT.....	2
CERTIFICATE	3
TABLE OF CONTENTS	4
LIST OF FIGURES.....	6
LIST OF TABLES.....	7
1. ABOUT WELLNIFY.AI	8
1.1. Platform Features	8
1.2. Leadership Team.....	9
2. EXECUTIVE SUMMARY	9
3. TEAM DYNAMICS AND RESPONSIBILITIES	11
4. TOOLS & TECHNOLOGIES.....	12
5. PROJECT OVERVIEW	13
5.1. User Engagement Dashboards	13
5.2. Recommendation System for User Engagement.....	14
6. PROJECT GOALS	15
7. LEARNING GOALS.....	17
7.1. Technical Learning Goals	17
7.2. Non-Technical Learning Goals	18
8. REQUIREMENTS ELICITATION	18
8.1. Stakeholder Identification	19
8.2. Techniques Used for Requirement Elicitation	19
8.3. Project Requirements	20
8.3.1. User Engagment Dashboards	20
8.3.2. Recommendation System for User Engagement	21
8.4. Requirement Documentation	23
9. METHODOLOGIES	23
9.1. User Engagement Dashboards	23
9.1.1. Data Collection and Pre-processing.....	24
9.1.2. Data Analysis.....	24
9.1.3. Backend Development.....	24

9.1.4. Frontend Development	25
9.2. Recommendation System for User Engagement.....	25
9.2.1. Data Collection, Cleaning, and Pre-processing.....	25
9.2.2. Model Development and Evaluation	26
9.2.3. Backend Integration	27
9.2.4. Iterative Model Improvement and Maintenance.....	27
10. IMPLEMENTATION	28
10.1. User Engagement Dashboards	28
10.1.1. Data Extraction and Initial Processing	28
10.1.2. Feature Engineering for Dashboard Metrics	29
10.1.3. Aggregation for Time-Based Metrics.....	30
10.1.4. Backend Integration and API Development	31
10.1.5. System Architecture	32
10.2. Recommendation System for User Engagement	33
10.2.1. Data Extraction and Initial Processing	34
10.2.2. Feature Engineering and Data Transformation.....	34
10.2.3. Model Development and Similarity Calculations	36
10.2.4. Backend Integration and API Development	37
10.2.5. Interaction Tracking and Continuous Improvement	38
11. WEEKLY IMPLEMENTATION BREAKDOWN	40
11.1. Week 1	40
11.2. Week 2	41
11.3. Week 3	42
11.4. Week 4	42
11.5. Week 5	43
11.6. Week 6	44
11.7. Week 7	45
11.8. Week 8	46
11.9. Week 9	47
11.10. Week 10	48
11.11. Week 11	49

11.12. Week 12	49
11.13. Week 13	50
11.14. Week 14	51
11.15. Week 15	53
11.16. Week 16	54
11.17. Week 17	56
11.18. Week 18	57
11.19. Week 19	59
11.20. Week 20	60
11.21. Week 21	62
11.22. Week 22	64
11.23. Week 23	65
11.24. Week 24	67
11.25. Week 25	68
12. CONCLUSIONS.....	69
12.1. Learning Goals Overview	70
12.2. Timesheet	71
13. ACHIEVEMENTS	72
14. CHALLENGES	73
15. FUTURE WORK	75
15.1. User Engagement Dashboards	75
15.2. Recommendation System for User Engagement	75
16. APPENDIX.....	77
16.1. References	77

LIST OF FIGURES

Figure 1: System Architecture Diagram	32
Figure 2: Data Processing Flowchart	35
Figure 3: Feedback Loop	38

LIST OF TABLES

Table 1: Learning Goals Overview	71
Table 2: Timesheet.....	71

1. ABOUT WELLNIFY.AI

Wellnify.ai is a leading wellness engagement platform that uses gamification and behavioral science to promote healthier lifestyles within communities. Founded by industry experts from successful sports technology start-ups Kinduct Technologies and TheProgram.ai, Wellnify.ai aims to transform community wellness engagement through fun, interactive, and rewarding experiences. The mission of Wellnify.ai is to inspire and empower communities and youth to lead healthier, more active lives. The company is dedicated to fostering positive change by providing access to resources, education, and opportunities that enhance physical fitness, mental well-being, and overall wellness. Their vision is to build resilient communities where everyone can achieve their full potential and lead a balanced life (Wellnify, 2024).

1.1. Platform Features

1. **Gamification:** Wellnify.ai incorporates elements of mobile gaming, such as challenges, badges, and augmented reality experiences, to motivate users and make wellness activities more enjoyable. This approach helps increase user engagement and fosters a sense of community
2. **Customizable Content:** Organizations can fully customize the platform to reflect their branding and specific needs. This includes uploading logos, selecting color schemes, and creating tailored wellness programs. The platform also allows for scheduling and delivering workouts, challenges, educational content, and wellness-centric events.
3. **Community Engagement:** The platform is designed to promote social interaction and competition among users. Individuals can connect with friends, colleagues, or community members and participate in wellness challenges together. This social aspect is key to maintaining ongoing engagement and motivation.
4. **Target Audience:** Wellnify.ai serves a diverse range of users, including corporations, community organizations, youth groups, and educational institutions. By promoting a culture of wellness and preventive healthcare, the platform aims to reduce healthcare costs and improve overall community health.

1.2. Leadership Team

The leadership team at Wellnify.ai includes:

- Travis McDonough: Founder & CEO, former Founder and CEO of Kinduct.
- Brendon Ferguson: Co-Founder & COO, former CEO of TheProgram.ai.
- Jeff Johnson: Co-Founder & VP Business Development, 14-year CFL veteran.
- Duncan Addison: Co-Founder & Director of Strategic Partnerships, former NCAA athlete.

Together, they bring over 100 years of combined experience in wellness technology, positioning Wellnify.ai to make a significant social and community impact (Wellnify, 2024).

Wellnify.ai has officially launched and is already operational in its core markets. The platform supports universities, corporations, healthcare systems, and youth organizations across North America, helping these communities engage members in wellness behaviors and promote healthier lifestyles (Wellnify, 2024).

2. EXECUTIVE SUMMARY

This report provides a comprehensive overview of my contributions as a Data Analyst and Backend Software Engineer at Wellnify.ai. Tasked with enhancing the platform's data analysis and visualization capabilities, my role spanned two pivotal projects: the development of User Engagement Dashboards and the implementation of a Recommendation System for User Engagement. These projects were instrumental in leveraging vast amounts of accumulated data to boost user interaction and engagement through advanced analytical tools and personalized content.

The first project focused on transforming Wellnify.ai's extensive data into actionable insights through sophisticated dashboards. This involved rigorous data cleaning to ensure integrity, employing statistical methods for deep analysis, and leveraging advanced Python libraries such as NumPy, Matplotlib, and Pandas. The integration of React technology facilitated the creation of dynamic, interactive dashboards that enhanced data accessibility and usability for stakeholders and users alike.

The second project entailed developing a machine learning-based recommendation system designed to personalize user interactions on the platform. This system used algorithms to analyze user behavior and preferences, enabling the delivery of tailored content and recommendations. My role included the backend implementation of this system, ensuring seamless integration and real-time data processing, enhancing the overall user experience and engagement.

The structured development cycle for both projects included phases such as requirements gathering, systematic planning, implementation of backend enhancements, and iterative development of dashboards and recommendation algorithms. Collaboration and effective teamwork were crucial, involving regular interactions with other software engineers and stakeholders to navigate challenges and align project outcomes with Wellnify.ai's strategic goals.

Despite the project's successes, there are several avenues for future enhancements to further leverage Wellnify.ai's technology. Future initiatives could include integrating more diverse data sources, such as real-time health metrics and user-generated content, to enrich the analytical depth of the platform. Additionally, ongoing refinement of the platform based on user feedback is crucial. This should focus on enhancing the user interface and functionality of the dashboards and recommendation systems to ensure they remain intuitive and user-friendly. Regular updates to the analytics models, based on new data and evolving user needs, will help maintain the relevance and effectiveness of Wellnify.ai's offerings.

In conclusion, this report not only details the objectives, methodologies, and achievements of the projects I led but also underscores the broader impact of my role in driving forward Wellnify.ai's data-driven initiatives within the wellness technology sector.

This report adheres strictly to the Non-Disclosure Agreement (NDA) signed with Wellnify.ai, ensuring the confidentiality of all proprietary information.

3. TEAM DYNAMICS AND RESPONSIBILITIES

During my tenure at Wellnify.ai, I worked as part of a dedicated team that included business analysts and full-stack developers. Each week, we held virtual team meetings every Tuesday to discuss weekly updates and progress, as well as regular virtual meetings with stakeholders, including Chief Technical Officer and occasionally the COO of the company, to review our progress and receive feedback.

As a Data Analyst and Software Engineer, my responsibilities included:

- **Understanding Existing Structures:** Analyzing the pre-existing application and data structures to determine optimal strategies for data analysis, enhancing user engagement, and meeting stakeholder requirements.
- **Data Insight Generation:** Creating new database tables to store valuable insights derived from data analysis.
- **Data Analysis and Pre-processing:** Utilizing Jupyter Notebook for initial data analysis, cleaning, and pre-processing with Python libraries such as NumPy, Pandas, and Matplotlib, and migrating the processed data to the database.
- **API Development:** Developing REST APIs to facilitate the display of interactive dashboards within the web application.
- **Quality Assurance:** Conducting thorough testing and performing bug fixes to ensure the robustness and reliability of the system.
- **Requirement Communication:** Collaborating effectively with the team lead to understand project requirements and provide regular feedback.
- **Frontend Collaboration:** Working closely with frontend developers to ensure seamless integration between backend services and frontend interfaces.
- **System Architecture Planning:** Designing and planning the architecture for the updated system to enhance its overall functionality and performance.

- **Model Development and Similarity Analysis:** Implementing the recommendation model using ML techniques, primarily focused on natural language processing (NLP) and similarity analysis. This involved using TfidfVectorizer for converting tag data into numerical vectors and cosine_similarity to compute similarity scores, enabling personalized item recommendations based on user interests.
- **Model Training and Evaluation:** Training the recommendation model with existing user interaction data and evaluating its performance. This included setting up cross-validation procedures to assess model accuracy and relevance, as well as tuning model parameters to improve recommendation quality. Regularly monitoring and iterating on the model based on feedback and test results.

These responsibilities required a blend of technical expertise, analytical acumen, and strong communication skills to ensure the successful completion of the project and the delivery of high-quality insights and visualizations to stakeholders.

4. TOOLS & TECHNOLOGIES

During my internship, Throughout the process of analyzing data, generating insights, and creating the web platform, several technologies and tools were utilized to ensure efficiency and effectiveness. The key tools, technologies, and libraries employed were:

- **Jupyter Notebook:** Used for initial data analysis, cleaning, and pre-processing. It provided an interactive environment for running Python code, visualizing data, and documenting the analysis process.
- **Python:** The primary programming language used for data analysis and backend development. Key libraries included NumPy for numerical computations, Pandas for data manipulation, and Matplotlib for data visualization.
- **Flask:** A lightweight web framework used to develop the backend services and REST APIs, facilitating communication between the database and the frontend.

- **React:** A JavaScript library for building user interfaces, used to create dynamic and responsive frontend components for the web application.
- **MySQL:** The relational database management system used to store, manage, and retrieve the data. It played a crucial role in organizing the data and supporting the backend services for the web application.

5. PROJECT OVERVIEW

The projects undertaken at Wellnify.ai aimed to enhance the app's functionality and user engagement by building tools for data analysis and user personalization. Two key initiatives were pursued to drive data-driven decision-making and improve user experience: a user engagement dashboard platform and a machine learning-based recommendation system. These projects provided stakeholders with actionable insights and personalized content suggestions, aligning with the company's goal of promoting healthier lifestyles through engaging and relevant user interactions.

5.1. User Engagement Dashboards

This project aimed to evaluate the impact and usefulness of the Wellnify app and enhance the company's decision-making strategies. Given the app's deployment across multiple organizations, it was essential to analyze vast amounts of data and present it in an accessible manner. The primary objective was to create a web platform to display meaningful insights and key statistics, enabling organizations to develop data-driven strategies.

The project lifecycle involved data collection and pre-processing, analysis, insights generation, and the creation of a user-friendly web platform. Initially, Jupyter Notebook was used to develop Python scripts for data analysis, cleaning, and pre-processing. These scripts generated statistics and migrated them to the respective databases. The web application was built using Flask and React, resulting in accessible and visually appealing dashboards.

The project's goal was to enhance user engagement and provide stakeholders with actionable insights. Data analysis and metrics were initially created using the development database, tested

in the staging database, and finally deployed to the production database to ensure accuracy and reliability.

Key technologies used included Python (with libraries such as NumPy, Pandas, and Matplotlib) for data analysis, Flask for backend development, and React for frontend development. Bootstrap and HighCharts were used for responsive web design and interactive charts, respectively, while MySQL managed and organized the data.

The development process involved understanding existing structures, generating new tables with insights, creating REST APIs, conducting thorough testing, and planning the system architecture for improved functionality and performance. This comprehensive approach ensured the delivery of high-quality insights and visualizations to stakeholders, meeting the project's objectives.

5.2. Recommendation System for User Engagement

The recommendation system project aimed to develop and implement a machine learning-based recommendation engine to provide personalized content suggestions within the app. Given the app's broad deployment across multiple organizations, it was essential to design a system capable of analyzing extensive user data and delivering accurate, data-driven recommendations that enhance user engagement. The primary objective was to integrate a recommendation pipeline that processes tag data and generates relevant suggestions through an API, supporting user experience and informing engagement strategies.

The project lifecycle included data preparation, model development, backend API integration, and iterative refinement. In the initial phase, Python and Jupyter Notebook were used to develop scripts for data preprocessing and to prototype the recommendation algorithms. These scripts cleaned and processed tag data, transforming it into vectors using TF-IDF for similarity analysis. A trial-and-error approach was essential during model development, with iterative tuning of the machine learning code to improve accuracy and relevance. As testing progressed, the model evolved, incorporating adjustments based on validation results, which ultimately led to a stable and accurate algorithm.

The recommendation engine was integrated with a backend built on Flask, which provided an API layer for real-time data retrieval and recommendation delivery within the app. This API interacted with a MySQL database, where tag-based data and recommendation results were stored, creating an end-to-end pipeline that included data collection, preprocessing, recommendation generation, and API response. This integration enabled the recommendation system to dynamically access user data and provide personalized suggestions in real time.

The combined efforts in developing user engagement dashboards and a recommendation system produced a robust, data-informed platform that empowers organizations to make strategic decisions while also enhancing individual user experiences. The dashboards offer insightful metrics on user behaviors, making data more accessible for stakeholders, while the recommendation engine delivers tailored content suggestions to foster ongoing engagement. Together, these projects underscore a commitment to utilizing advanced analytics and machine learning to create an impactful, user-centered experience within the app, reinforcing the company's mission to drive positive wellness outcomes.

6. PROJECT GOALS

The primary goals of the Wellnify.ai project were as follows:

- **Assess User Engagement and Data Utilization:** To analyze the app's data to gain insights into user behaviors and engagement patterns, providing a comprehensive understanding of how the app is used and identifying areas for improvement.
- **Enhance Decision-Making Strategies:** To provide Wellnify and its stakeholders with actionable insights through comprehensive data analysis, aiding in informed decision-making and strategy development.
- **Create a User-Friendly Web Platform:** To develop an accessible and visually appealing web platform that displays meaningful insights and key statistics, making it easier for organizations to understand and utilize their data.

- **Optimize Data Analysis Processes:** To implement robust data analysis and preprocessing techniques, ensuring data integrity and generating reliable insights. This involved using Python and its libraries, such as NumPy, Pandas, and Matplotlib, for data manipulation and visualization.
- **Develop a Recommendation System for Personalized User Engagement:** To design and implement a recommendation engine that leverages machine learning to deliver personalized content suggestions based on user activity and tag data. This system aimed to improve user engagement by providing relevant, tailored experiences and supporting the app's goal of encouraging active participation.
- **Implement and Refine Machine Learning Models:** To develop, test, and refine machine learning models using a trial-and-error approach, ensuring that the recommendation algorithm effectively adapts to user data and delivers accurate, meaningful recommendations. This iterative approach helped achieve optimal performance and relevance in content suggestions.
- **Develop and Integrate APIs:** To create REST APIs using Flask for seamless communication between the backend and the frontend, supporting the integration of interactive dashboards.
- **Enhance System Architecture:** To design and plan an optimized system architecture that supports real-time data processing and visualization, ensuring scalability and performance.
- **Conduct Thorough Testing and Quality Assurance:** To perform extensive testing and bug fixing to ensure the reliability and performance of the system, adhering to industry standards.

By achieving these goals, the project aimed to transform raw data into valuable insights, enhance user engagement through personalized recommendations, and provide Wellnify and its stakeholders with a powerful tool for data-driven decision-making.

7. LEARNING GOALS

During my tenure at Wellnify.ai, I set several technical and conceptual learning goals to enhance my skills and knowledge as a Data Analyst and a Software Engineer. These goals aimed at both technical proficiency and a deeper understanding of the wellness technology industry.

7.1. Technical Learning Goals

- **Master Python Programming:** Develop extensive proficiency in Python, focusing on advanced features and best practices. Gain in-depth knowledge of key Python libraries such as NumPy, Pandas, and Matplotlib for data analysis, as well as Flask for web development.
- **Understand Data Analysis Techniques:** Learn thorough data cleaning, pre-processing, and analysis techniques to extract meaningful insights from large datasets.
- **Develop Skills in REST API Development:** Acquire hands-on experience in creating and managing REST APIs using Flask to support the web application's backend.
- **Explore Database Management:** Understand MySQL database management, including data migration and integration with web applications.
- **Build Proficiency in Machine Learning for Recommendations:** Gain hands-on experience in developing machine learning models, specifically recommendation algorithms. Focus on implementing similarity measures such as TF-IDF vectorization and cosine similarity to deliver personalized recommendations.
- **Model Tuning and Evaluation:** Develop skills in model tuning, validation, and evaluation to optimize recommendation algorithms. Learn to use cross-validation techniques and performance metrics to refine models for accuracy and relevance in real-world applications.
- **Learn Feature Engineering Techniques:** Gain expertise in feature extraction and engineering, specifically for transforming raw text data into structured features that can

be used for machine learning models. This includes applying text vectorization techniques and handling multi-label data effectively.

- **Implement Iterative Model Development:** Develop an understanding of iterative model development, including trial-and-error approaches to refine machine learning models. Learn to experiment with parameters and techniques systematically to achieve desired outcomes.
- **Software Development Lifecycle (SDLC):** Gain a comprehensive understanding of the SDLC, including phases such as planning, analysis, design, implementation, testing, deployment, and maintenance. Learn to apply these principles in developing robust and efficient software solutions.

7.2. Non-Technical Learning Goals

- **Understand the Wellness Technology Industry:** Gain a comprehensive understanding of the wellness technology sector, focusing on how technology can enhance user engagement and improve health outcomes.
- **Learn About User Engagement Strategies:** Study various strategies to boost user engagement through data-driven insights and personalized content.
- **Enhance Communication Skills:** Develop effective communication skills to collaborate with team members and stakeholders, ensuring clear and concise information exchange.
- **Understand Business Processes:** Learn about the business processes at Wellnify.ai, including project management and workflow optimization.

By achieving these learning goals, I aimed to significantly contribute to the project's success while enhancing my technical and professional skills in the field of data science and software development.

8. REQUIREMENTS ELICITATION

Requirement elicitation is a critical phase in the software development lifecycle (SDLC) that involves gathering and defining what the stakeholders need from the project. For the Wellnify.ai

project, this process was particularly important to ensure that the developed solution met the needs of various organizations and their users. The goal was to create a web platform that provided actionable insights through data analysis and interactive dashboards , and a machine learning-based recommendation model.

8.1. Stakeholder Identification

The first step in the requirement elicitation process was identifying the key stakeholders. This included:

- Internal Stakeholders:
 - CTO
 - COO
 - Team leads and developers at Wellnify.ai
- External Stakeholders:
 - Organizational clients using the Wellnify app
 - End-users of the Wellnify app

8.2. Techniques Used for Requirement Elicitation

To gather comprehensive and accurate requirements, the following elicitation techniques were employed:

1. **Initial Meetings:** Conducted a couple of initial meetings with the CTO and COO where they explained the app, provided a demo, and highlighted their pain points.
2. **User Access and Observation:** Gained access to the app as users to observe how end-users interact with it. This helped in understanding user behavior and identifying areas for improving engagement and insights.
3. **Weekly Meetings:** Held regular weekly meetings with the CTO, Mr. Rob Moore and the internal team to discuss progress, share insights, and ensure alignment among all team members.

8.3. Project Requirements

8.3.1. User Engagment Dashboards

Based on the requirement elicitation process, the following key requirements were identified for the dashboard project:

- **Data Extraction and Analysis:**
 - Develop methods to extract data from the existing MySQL database.
 - Implement data cleaning and pre-processing techniques to ensure data integrity.
 - Use Python and its libraries (NumPy, Pandas, Matplotlib) for comprehensive data analysis.
- **Insight Generation:**
 - Generate meaningful insights from the analyzed data.
 - Create new database tables to store these insights, ensuring they are accessible for further processing.
- **Interactive Dashboards:**
 - Design and develop interactive dashboards to display the generated insights.
 - Use Flask for backend development and React for frontend development to ensure a seamless and responsive user experience.
 - Integrate visualization tools like HighCharts and Bootstrap to enhance the dashboards' visual appeal.
- **REST API Development:**
 - Develop REST APIs to facilitate communication between the backend and the frontend, ensuring efficient data retrieval and presentation.
- **Performance and Scalability:**
 - Optimize backend processes to ensure the system can handle large volumes of data and user requests efficiently.
 - Ensure the platform is scalable to accommodate future growth in data and user base.

- **Security and Compliance:**
 - Implement robust security measures to protect sensitive user data.
 - Ensure compliance with relevant data protection regulations and standards.
- **User Engagement and Feedback:**
 - Continuously gather feedback from users and stakeholders to refine and improve the platform.
 - Implement features to enhance user engagement based on the observed interaction patterns and feedback.

8.3.2. Recommendation System for User Engagement

Based on the requirement elicitation process, the following key requirements were identified for the recommendation system project:

- **Data Extraction, Transformation, and Analysis:**
 - Develop methods to extract user tag data and activity information from the existing MySQL database to provide input for the recommendation algorithm.
 - Implement data cleaning, transformation, and pre-processing techniques, particularly for handling and structuring tag data, to ensure data integrity and consistency.
 - Use Python libraries, such as pandas for data manipulation, TfidfVectorizer for text transformation, and cosine_similarity for similarity calculations, to develop a reliable and effective recommendation pipeline.
- **Recommendation Model Development:**
 - Build a machine learning model to suggest personalized workout plans based on user data, with a focus on identifying relevant items or activities for each user.
 - Apply TF-IDF vectorization and cosine similarity to compute content similarities and determine recommendations.

- Continuously refine the model through a trial-and-error approach, adjusting parameters, similarity thresholds, and vectorization settings to optimize model accuracy and recommendation quality over time.
- **Recommendation API and Backend Integration:**
 - Design and implement REST APIs using Flask to integrate the recommendation engine with the backend, allowing real-time data processing and delivery of personalized recommendations.
 - Structure the recommendation API to support efficient data retrieval and real-time responses, ensuring that recommendations are updated based on user interactions or data changes.
 - Maintain compatibility with the MySQL database, storing computed recommendations and user engagement data in dedicated tables for easy access and future analysis.
- **Performance and Scalability:**
 - Optimize the recommendation pipeline and backend processes to handle large data volumes and support real-time recommendation generation without impacting system performance.
 - Ensure the model is scalable to accommodate growth in user data and additional recommendation features, allowing for future expansion and updates.
- **Model Validation and Continuous Improvement:**
 - Regularly validate model accuracy and relevance through A/B testing, cross-validation, and performance evaluations to ensure accurate high-quality recommendations.
 - Gather user feedback and monitor engagement metrics to refine and improve the recommendation algorithm based on observed outcomes and stakeholder input.
 - Implement a structured process for model updates, allowing adjustments to the algorithm as new data or user trends emerge, thus ensuring the recommendation system remains relevant and effective over time.

8.4. Requirement Documentation

The gathered requirements were meticulously documented to ensure clarity and traceability throughout the project. The documentation included:

- **Functional Requirements:**
 - Detailed specific functionalities needed, such as data extraction, analysis processes, and the creation of REST APIs for the web application.
 - Defined the features of the interactive dashboards, including data visualization types and user interaction capabilities.
 - Outlined the requirements for the recommendation system, specifying the use of machine learning algorithms for personalized content suggestions, model integration with the backend, and API endpoints for recommendation delivery.
- **Mockups and Prototypes:**
 - Created visual prototypes of the dashboards and key interfaces to provide stakeholders with a tangible representation of the final product.
 - Developed mockups for the recommendation feature, illustrating how personalized suggestions would be displayed within the app interface.
 - Iteratively refined these prototypes based on stakeholder feedback.

9. METHODOLOGIES

9.1. User Engagement Dashboards

This project followed a structured and technical approach to ensure the successful development and implementation of a data-driven web platform. The methodologies employed spanned multiple phases of the software development lifecycle (SDLC), focusing on data extraction, analysis, backend and frontend development, and quality assurance.

9.1.1. Data Collection and Pre-processing

- **Data Extraction:** The initial step involved extracting data from the existing MySQL database. SQL queries were used to fetch relevant data, ensuring only the necessary information was retrieved to optimize performance and reduce data redundancy.
- **Data Cleaning:** Data cleaning was performed using Python libraries such as Pandas. This process involved handling missing values, correcting inconsistencies, and removing duplicate entries to ensure data integrity.
- **Data Transformation:** Transformation operations were applied to convert the raw data into a suitable format for analysis. This included normalizing numerical values, categorizing data, and creating new derived attributes to facilitate deeper insights.

9.1.2. Data Analysis

- **Exploratory Data Analysis (EDA):** EDA was conducted using Jupyter Notebook to understand data distributions, relationships, and trends. Visualization tools such as Matplotlib and Seaborn were used to generate plots and graphs, aiding in identifying patterns and anomalies.
- **Statistical Analysis:** Statistical techniques, including hypothesis testing and correlation analysis, were employed to validate assumptions and derive meaningful insights. This process was critical in ensuring the reliability and significance of the findings.

9.1.3. Backend Development

- **API Development:** RESTful APIs were developed using Flask to facilitate seamless communication between the backend and frontend. These APIs handled data requests, processing, and delivery of insights to the web application.
- **Database Management:** The insights generated from data analysis were stored in new MySQL tables. Database management practices, such as indexing and query optimization, were implemented to enhance data retrieval performance.

- **Data Migration:** Python scripts were developed to automate the migration of processed data into the database. This ensured that the insights were consistently updated and available for visualization.

9.1.4. Frontend Development

- **User Interface Design:** The frontend was developed using React to create a dynamic and responsive user interface. The design focused on user experience (UX) principles, ensuring the dashboards were intuitive and easy to navigate.
- **Visualization Integration:** Visualization libraries such as HighCharts were integrated to create interactive and customizable charts and graphs. These visualizations enabled users to explore data insights effectively.
- **Responsive Design:** Bootstrap was used to ensure the web application was responsive and accessible across different devices and screen sizes.

By following these methodologies, I successfully developed a robust and efficient web platform that provided meaningful insights and enhanced user engagement. This structured approach ensured that each phase of the project was executed with precision, resulting in a high-quality solution tailored to the needs of Wellnify and its stakeholders.

9.2. Recommendation System for User Engagement

This project followed a structured and technical approach to ensure the successful development and implementation of a machine learning-based recommendation system. The methodologies employed spanned multiple phases of the software development lifecycle (SDLC), focusing on data extraction and transformation, model development and optimization, backend API integration, and continuous improvement through iterative testing.

9.2.1. Data Collection, Cleaning, and Pre-processing

- **Data Extraction:** The initial step involved extracting relevant data, specifically user tags and activity records, from the MySQL database. SQL queries were used to retrieve only

the necessary information for model input, minimizing data redundancy and optimizing processing efficiency.

- **Data Cleaning:** Data cleaning was performed using Python libraries such as pandas. This step handled missing values, corrected inconsistencies in tag structures, and removed duplicate entries to ensure data integrity. Given the variability in user tag data, particular attention was paid to standardizing formats for consistency.
- **Data Transformation:** Transformation operations were applied to prepare the raw data for machine learning. These operations included converting tags into numerical vectors using TF-IDF vectorization, creating structured features that capture the semantic relevance of each tag. Multi-label binarization was also applied where necessary, enabling efficient handling of multiple tags associated with individual users.

9.2.2. Model Development and Evaluation

- **Initial Model Prototyping:** The recommendation model was developed using a TF-IDF vectorizer to transform tag data into vectors and cosine similarity to measure the relevance between users and content. Initial prototypes were built in Jupyter Notebook, allowing for rapid testing of parameters and visualization of results.
- **Trial and Error for Model Optimization:** A trial-and-error approach was used to refine the model iteratively. Parameters such as TF-IDF thresholds and similarity scores were adjusted to improve accuracy and relevance. Various similarity thresholds and vector configurations were tested to identify the settings that produced the most reliable and contextually relevant recommendations.
- **Model Evaluation and Validation:** To ensure the model's effectiveness, cross-validation techniques were applied. Metrics such as precision, recall, and F1-score were used to evaluate recommendation accuracy and ensure the model consistently delivered valuable suggestions. A/B testing was also planned for post-deployment to measure user

engagement with recommendations and further refine the algorithm based on real-world feedback.

9.2.3. Backend Integration

- **API Development for Recommendation Delivery:** RESTful APIs were developed using Flask to facilitate the communication between the recommendation model and the frontend. The recommendation API provided endpoints for real-time retrieval of user-specific recommendations, ensuring a seamless and responsive experience.
- **Pipeline Development and Data Flow Management:** A robust data pipeline was created to handle real-time data flow from the MySQL database to the recommendation model and back to the frontend. The pipeline enabled efficient data retrieval, transformation, and recommendation generation, supporting real-time interactions.
- **Database Management:** New tables were created in MySQL to store model-generated recommendations, user engagement metrics, and interaction logs. Database optimization practices, including indexing and query refinement, were implemented to support efficient storage and retrieval of recommendation data.

9.2.4. Iterative Model Improvement and Maintenance

- **Model Monitoring and Logging:** The model's performance was continuously monitored, with logging implemented to capture data on user interactions, recommendation success rates, and potential anomalies. This logging system provided feedback for further adjustments, ensuring the recommendation engine remained effective as user preferences evolved.
- **Continuous Model Refinement:** The recommendation model was updated regularly based on engagement metrics and user feedback. This iterative refinement included adjustments to similarity thresholds, feature engineering, and the incorporation of new data points to improve recommendation accuracy.

This structured approach, combined with an iterative model improvement strategy, ensured that the recommendation system was robust, accurate, and effectively enhanced user engagement. Through backend integration and continuous monitoring, the system was maintained and improved over time, adapting to the evolving preferences and engagement patterns of users.

10. IMPLEMENTATION

10.1. User Engagement Dashboards

The implementation of my project involved developing a data-driven web platform to provide actionable insights and enhance user engagement. The system was designed to be independent of the existing Wellnify app, I worked with a development database which was deployed on AWS to ensure synchronicity among team members. Below is a detailed description of the implementation process and system architecture.

10.1.1. Data Extraction and Initial Processing

To ensure data consistency and accuracy, I began by setting up a connection to the database and systematically extracting data for analysis. Using SQLAlchemy and environment variables to securely load database credentials, I established an engine for database queries. Key steps in this phase included:

1. Data Extraction:

- I retrieved relevant data tables and filtered them to exclude any records flagged as deleted. This ensured that only active records were processed, minimizing data noise.
- Using SQL queries, I extracted raw activity logs, user data, and other pertinent metrics and stored them as pandas DataFrames, making them easier to manipulate and transform as needed.

2. Data Cleaning:

- Data cleaning involved handling missing values and standardizing date and time fields to support time-based analyses. I converted timestamps to a consistent format, allowing for accurate aggregations by day, week, and month.
- I applied custom functions to handle edge cases, such as missing or non-standard values, ensuring a consistent structure across all data sources.

10.1.2. Feature Engineering for Dashboard Metrics

The next phase involved deriving various metrics and features to provide a comprehensive overview of user engagement. These metrics formed the basis of the dashboard and were carefully calculated to ensure meaningful insights.

1. Activity Summary Metrics:

- I created summary metrics that aggregated user activity data, such as Total Attempted activities and Completion Rate, calculated as the percentage of completed activities relative to total attempts. These metrics offered a high-level view of user engagement and interaction quality.
- Additional metrics included Most Active Hour and Most Active Day, calculated using the mode of activity hours and days for each user. These metrics helped stakeholders understand peak engagement times, informing strategies for content delivery.

2. User-Specific Activity Scores:

- **Activity Value Score (AVS):** I calculated a weekly activity value score for each user, summarizing their engagement level over each week. By grouping data by week, I created a time-based view of engagement, capturing fluctuations in user activity.
- **Consistency Score (CS):** This score represented the number of active weeks, giving insights into user consistency. I normalized this score to provide a comparable

measure across users, scaling it between 50 and 100 to represent engagement levels.

- **Combined Engagement Score:** I computed a composite score by averaging the AVS and CS, offering a balanced measure of both activity value and consistency.

3. Peak and Consistency Metrics:

- **Highest Daily and Weekly Activity:** For each user, I calculated their highest daily and weekly activity levels, helping to identify users with significant spikes in engagement.
- **Most Consistent Month:** To identify periods of high activity, I calculated each user's most consistent month based on their activity frequency. This metric helped pinpoint when users were most engaged, supporting seasonal or campaign-driven engagement strategies.

4. Improvement Over Time:

- **Improvement Score:** I developed a monthly improvement score based on each user's average activity. By calculating and standardizing monthly activity values, I identified users with positive trends in engagement, reflecting improvements over time.
- **Amplified Improvement:** I applied an exponential function to the improvement score, which highlighted users with accelerating engagement trends, making it easier to identify users responding well to engagement efforts.

10.1.3. Aggregation for Time-Based Metrics

To support a time-based view of user engagement, I aggregated data at multiple temporal levels, including daily, weekly, and monthly. This allowed the dashboard to provide a granular view of user activity and track changes in engagement over time.

1. Daily, Weekly, and Monthly Activity Aggregation:

- I grouped data by user and day, week, or month, calculating the mean activity value for each period. These aggregates enabled stakeholders to track engagement trends at varying time scales.
- For each level of aggregation, I created separate tables in the database for daily, weekly, and monthly activities, enabling efficient access and query performance.

2. Streak Tracking:

- To capture user engagement consistency, I implemented logic to identify streaks of consecutive active days. By calculating daily gaps and identifying sequences without breaks, I determined the longest streak for each user, indicating sustained engagement.

3. Monthly Consistency:

- I also measured user consistency at the monthly level, calculating the number of active days within each month. This metric provided a broader view of user engagement trends, allowing the dashboard to highlight months with the highest sustained engagement.

10.1.4. Backend Integration and API Development

With the metrics and aggregated data prepared, I integrated these results into the backend for efficient delivery to the dashboard. This process involved creating new tables in the database to store precomputed metrics, ensuring the data was readily accessible for visualization.

1. Storage and Organization of Metrics:

- I stored each processed metric in its own table to simplify data access and optimize query performance. Key tables included those for user activity scores, daily/weekly/monthly activity aggregates, and improvement scores.
- Each table was indexed by user ID and relevant date fields (e.g., week or month) to support fast retrieval for specific users or time periods.

2. Automated Data Updates:

- Using a scheduled task, I automated the refresh of these metrics, ensuring the dashboard displayed up-to-date information. By setting regular intervals for data recalculations, I minimized processing load while keeping the metrics current.
- For real-time updates, I integrated a trigger mechanism to recalculate metrics on significant events (e.g., user activity), ensuring the dashboard stayed in sync with recent engagement patterns.

3. API Integration:

- I developed RESTful API endpoints using Flask to serve metrics to the dashboard. Each endpoint was designed to retrieve specific metrics (e.g., user engagement scores or peak activity metrics) and deliver them in JSON format, enabling efficient frontend integration.
- To reduce latency, I optimized API queries by retrieving precomputed metrics directly from the database tables, minimizing the need for additional processing.

10.1.5. System Architecture

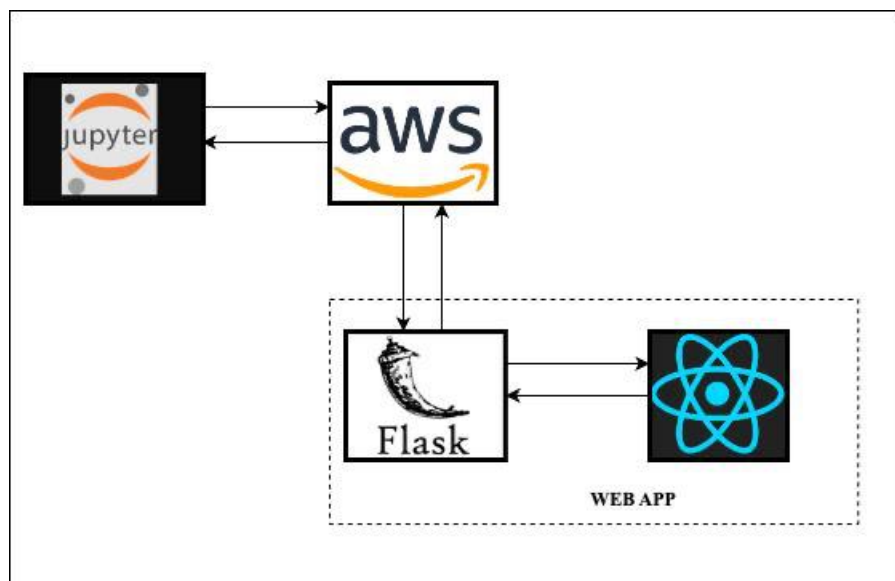


Figure 1: System Architecture Diagram

The architecture described in Figure 1 consisted of multiple components, each serving a specific function in the data processing and visualization pipeline:

- **Jupyter Notebook:** Used for initial data analysis, cleaning, and preprocessing. Python scripts were developed to handle various data operations and interacted with the AWS-hosted devdb to fetch and process data.
- **AWS:** Hosted the devdb, ensuring seamless access and synchronization for the development team. Provided scalable storage and computing resources for data processing tasks.
- **Flask:** Developed as an independent backend microservice, hosted inside a Docker container. Provided RESTful APIs for the web application, facilitating data retrieval and interaction. Included triggers to enable real-time data processing, calculating metrics as soon as users performed any activity on the app.
- **React:** Used to build the frontend of the web application. Created dynamic and responsive dashboards to display insights and statistics. Interacted with the Flask backend through the RESTful APIs.
- **Docker:** Containerized the Flask application to ensure consistent deployment and scalability. Allowed the microservice to be easily integrated with the existing Wellnify infrastructure.

10.2. Recommendation System for User Engagement

The implementation of the recommendation system was an in-depth process that involved building a comprehensive data pipeline, developing an effective machine learning model, and integrating it into the backend API for real-time recommendations. Each stage required careful planning, iterative development, and fine-tuning to achieve a system capable of delivering personalized, relevant content suggestions to users.

10.2.1. Data Extraction and Initial Processing

To initiate the recommendation pipeline, I began by setting up a robust data extraction process using SQLAlchemy to connect to the MySQL database. Key data elements used in the system included information on each content collection, mappings between collections and tags, links between activities and collections, and user-to-organization mappings to ensure organization-specific recommendations.

For data retrieval, I used SQL queries to load data into pandas DataFrames and stored them locally as CSV files for reference, ensuring data was readily accessible and easy to manipulate for analysis and model training. This allowed me to structure data into the formats required for feature engineering and model input.

Data formatting and handling missing values were necessary steps, especially for mappings between collections and tags, as well as links between activities and collections. To address these the following steps were performed:

- **Removed Duplicates and Nulls:** Applied data cleaning techniques using pandas to handle missing values and remove duplicates, ensuring that each collection had a unique set of tags and activities associated with it.
- **Standardized Tag and Activity Data:** I standardized tag formats by transforming them into lowercase and replacing spaces with underscores, ensuring uniformity across tags. This step was essential for later similarity calculations.

10.2.2. Feature Engineering and Data Transformation

With clean data in place, I transformed it into a format suitable for similarity-based recommendations. This step included encoding categorical features and extracting meaningful information from textual data.

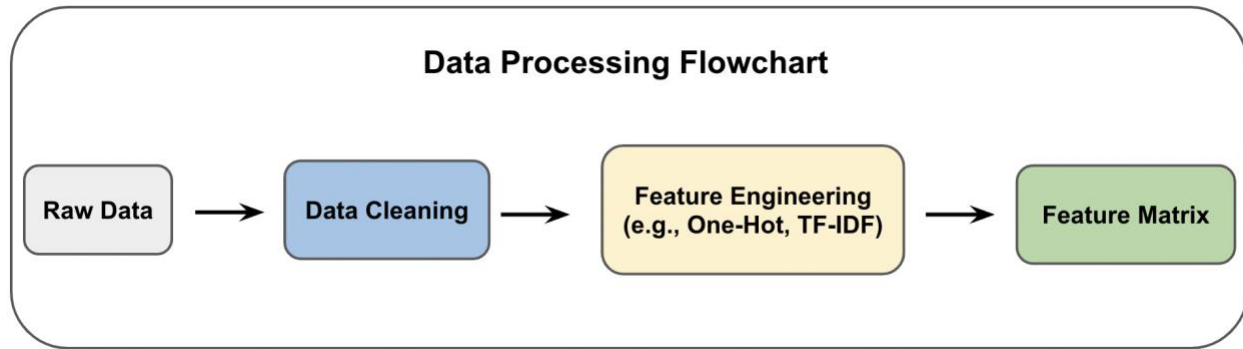


Figure 2: Data Processing Flowchart

1. One-Hot Encoding for Categorical Features:

- **Tags:** I applied one-hot encoding to each tag to create a binary matrix, where each column represented a specific tag, and each row represented a collection. This matrix allowed the model to interpret the presence or absence of tags as a feature, enhancing content relevance based on tag similarity.
- **Collection Type:** I performed one-hot encoding on collection type, capturing the type of each collection. This feature helped distinguish collections by content type, enabling the model to give appropriate weight to similarities in type.
- **Activities:** Activity data linked to each collection was also one-hot encoded. This matrix provided insights into the kinds of interactions collections had, further refining similarity calculations.

2. Text Processing with TF-IDF Vectorization:

- I combined relevant text fields (e.g., title and content summary) into a single column, capturing both high-level and detailed descriptions of each collection.
- To convert this textual data into numerical format, I used `TfidfVectorizer` from Scikit-Learn, transforming the text into a sparse matrix where each entry represented the importance of a word across all collections. The TF-IDF matrix allowed the model to assess content relevance based on term frequency, emphasizing unique terms within each collection.

3. Sparse Matrix Creation for Efficient Computation:

- After transforming both categorical and textual features, I combined these matrices into a single sparse matrix using SciPy's `hstack` function. Sparse representation was essential for handling high-dimensional data without compromising memory efficiency.

These feature transformations were pivotal in creating a unified feature matrix that allowed the recommendation model to leverage both categorical and textual information. The feature matrix was then standardized to a consistent format, ensuring uniformity across all features.

10.2.3. Model Development and Similarity Calculations

The recommendation model relied on a similarity-based approach, where collections similar to those a user had interacted with were recommended. The development of this model involved careful calculation of cosine similarity and iterative adjustments to optimize relevance:

1. Cosine Similarity Calculation:

- I computed cosine similarity across the entire feature matrix using Scikit-Learn's `cosine_similarity` function. This method quantified the similarity between collections, based on both content features and user interaction data.
- I computed similarity matrices for individual feature groups (e.g., tags, collection type, activities) and combined these to form a final similarity matrix. This structure allowed flexibility in weighting different features during recommendation generation.

2. Iterative Tuning and Weight Adjustment:

- Through extensive testing, I adjusted the weights for different features, including textual, tag-based, and activity-based similarities. For instance, TF-IDF vectorization for textual data was initially weighted at a certain level, while tag similarity was given another weight. These weights were fine-tuned by observing changes in recommendation accuracy.

- I applied a trial-and-error approach to refine the model's performance, running validation tests to evaluate the impact of each feature type on recommendation quality. This iterative tuning helped balance the influence of various feature groups, ensuring that recommendations were well-rounded and relevant to user preferences.

3. Tag Weighting with Smoothed Inverse Frequency:

- To avoid bias from frequently occurring tags, I implemented a smoothed inverse frequency technique, assigning lower weights to common tags. Using a smoothing factor, I adjusted tag weights so that infrequent tags received more emphasis, promoting diverse recommendations. This adjustment proved effective in delivering personalized results by reducing the impact of generic tags.

10.2.4. Backend Integration and API Development

With the model trained and optimized, I integrated it into the backend through a RESTful API developed using Flask. This integration allowed real-time generation of recommendations based on live user interactions. The steps for backend integration included:

1. Recommendation API Endpoint:

- I created an API endpoint within Flask, designed to accept a user ID as input and return a set of recommended collections. The function retrieved collections that the user had previously interacted with, calculated similarity scores using the precomputed cosine similarity matrix, and filtered out previously interacted collections.
- The API was designed to be efficient and responsive, querying only the necessary portions of the similarity matrix for each user request. This ensured minimal latency and high scalability, even as the number of users increased.

2. Dynamic Data Retrieval and Filtering by Organization:

- Recommendations were filtered to include only those collections associated with the user's organization(s), as retrieved from organizational mappings. This organization-based filtering enabled personalized recommendations aligned with the user's community, improving contextual relevance.
- I used dictionary mappings to quickly locate the relevant indices in the similarity matrix, further optimizing the retrieval process.

3. Storage and Logging:

- Recommendations and engagement metrics (such as click rates) were stored in MySQL for future analysis and feedback-based model improvements. This logging system captured each recommendation and the user's response, enabling analysis of recommendation quality over time.

10.2.5. Interaction Tracking and Continuous Improvement

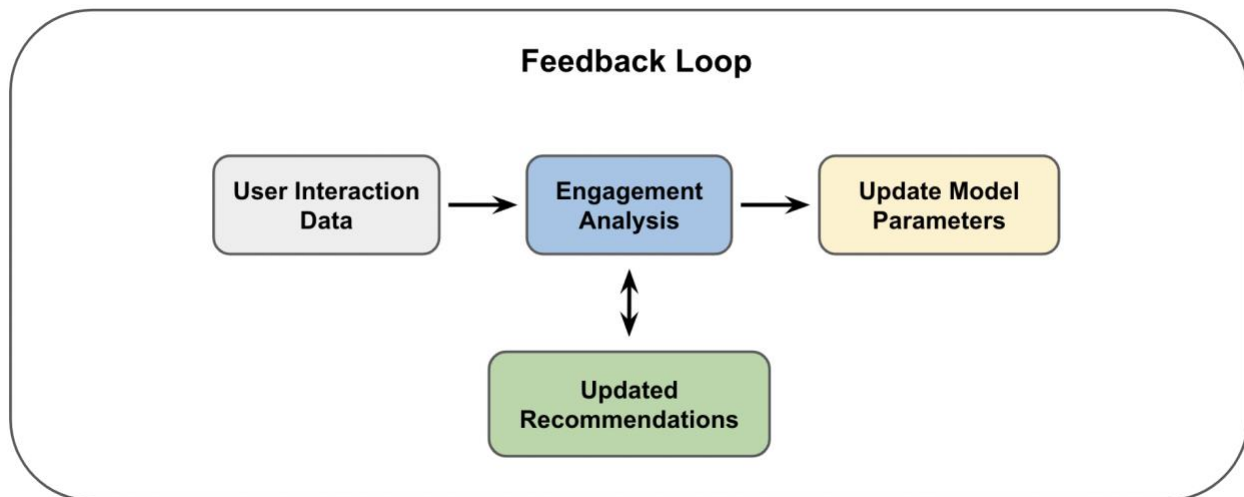


Figure 3: Feedback Loop

To improve the recommendation model continuously, I integrated user feedback mechanisms that captured interaction data.

1. User Interaction Logging:

- Each user interaction with a recommended collection was logged, capturing engagement metrics such as click-through rates and time spent on recommended content. This data was stored for subsequent analysis, providing a feedback loop for evaluating recommendation accuracy.
- Interaction logs were analyzed periodically to identify patterns in user engagement. For instance, if a specific tag or collection type consistently generated high engagement, I used this insight to adjust feature weights or model parameters.

2. A/B Testing for Model Refinement:

- I implemented A/B testing to assess the impact of different configurations on user engagement. By comparing recommendation performance across various feature weightings, I was able to determine which configurations yielded the highest engagement and relevance.
- A/B testing results informed adjustments in the recommendation algorithm, allowing the system to evolve in response to changing user preferences.

By following this detailed, structured approach, I implemented a reliable, high-performance recommendation system that dynamically adapts to user behavior. The continuous improvement mechanisms and robust backend integration allow the system to deliver high-quality, personalized recommendations at scale, enhancing user engagement and supporting data-driven insights. This comprehensive methodology, encompassing data preparation, iterative tuning, backend API development, and quality assurance, ensures the recommendation system remains responsive, relevant, and effective.

11. WEEKLY IMPLEMENTATION BREAKDOWN

11.1. Week 1

During Week 1, I laid the groundwork for my project with Wellnify, starting with an official meeting with the company's COO, Mr. Brendon Fergusen, and CTO, Mr. Rob Moore. This initial meeting was crucial in setting the stage for our collaboration and aligning our objectives with Wellnify's mission and goals.

Official Meeting with Wellnify's COO and CTO: The meeting began with introductions, ensuring that all team members and stakeholders were well-acquainted. This helped me in fostering a collaborative atmosphere right from the start. We discussed Wellnify.ai's foundation, goals, and mission, gaining a comprehensive understanding of the company's vision. I reviewed the presentation that Wellnify shares with prospective clients, which provided insights into the company's market positioning and value proposition. Following this, a comprehensive demo of the current Wellnify app was provided, showcasing its features, user interface, and functionality. I also discussed the current app limitations and the scope of planned improvements, identifying key areas for enhancement.

App Overview: Access to the Wellnify app was granted, allowing me to explore its features and functionalities in-depth. I spent time navigating the app to understand the user perspective. This hands-on exploration was critical in identifying potential improvements and noting down any feedback or changes needed. Additionally, I gained an understanding of the app's data structure, which would be essential for my analysis and subsequent development work.

Drafting an Initial Proposal: A follow-up call was held with team members to discuss potential areas to be explored. The discussion concentrated on key focus areas such as "Data Analysis," "Enhanced Insights," "Web App for displaying new insights," "Upgrades to the existing Dashboard," and "Activity-based Workout Recommendation Model." I explored various ideas for app enhancements, ensuring they were consistent with Wellnify's brand and met their requirements and expectations. This brainstorming session was instrumental in drafting an initial

proposal outlining various features and their implementation steps. The proposal aimed to provide a clear roadmap for our development efforts and align our goals with Wellnify's strategic objectives.

By the end of Week 1, I had established a solid foundation for the project, with a clear understanding of Wellnify's objectives, a thorough exploration of the app, and a well-drafted initial proposal. This preparation set the stage for the subsequent weeks of development and collaboration.

11.2. Week 2

Week 2 focused on finalizing the preliminary project plan, assigning tasks, and completing necessary onboarding processes.

Preliminary Project Plan: I started the week with a crucial meeting with Mr. Rob and the team to review the initial proposal drafted in Week 1. During this meeting, I discussed the feedback provided and additional requirements gathered, ensuring that the plan aligned with Wellnify's strategic goals. Based on the insights from this meeting, I developed a comprehensive preliminary project plan. This plan included a detailed task breakdown and role assignments among the three team members. It was decided that all three of us would contribute to the data analysis phase, leveraging our collective expertise to gain deeper insights. Additionally, I took on the responsibility for backend development, ensuring robust and scalable backend systems.

NDA Signing: As part of the onboarding process, I read and signed the Non-Disclosure Agreement (NDA). This step was essential to ensure the confidentiality and security of Wellnify's sensitive information. Signing the NDA reinforced our commitment to maintaining high standards of data protection and confidentiality throughout the project.

By the end of Week 2, I had a clear and structured project plan in place, with defined roles and responsibilities for each team member. Additionally, the NDA signing ensured that I was fully onboarded and ready to proceed with the project's next phases, maintaining the integrity and confidentiality required for successful collaboration.

11.3. Week 3

Week 3 focused on the initial database setup, exploring and understanding the development database, and tackling the challenges related to data analysis.

Initial Database Setup and Exploration: This week began by gaining access to the development database. This crucial step allowed me to load the database into my local workbench environments, where I could execute MySQL queries. These queries were essential for gaining an initial understanding of the database structure and its contents. By familiarizing myself with the data, I was better prepared to plan our subsequent analysis and development tasks.

Database Schema Design: As part of optimizing the database for our project needs, I implemented various database optimizations and indexing strategies. These measures were aimed at improving the performance and efficiency of the queries, ensuring that the database could handle the demands of our analysis and application processes.

Challenges in Data Understanding and Analysis: I encountered several challenges during this phase, primarily due to the complexity and unstructured nature of the development database. The database contained numerous tables with a significant amount of garbage values, making it difficult to derive meaningful insights. To address these challenges, I researched various methods to understand and analyze the data using Python. This involved exploring different data analysis techniques and tools that could help me clean, structure, and interpret the data effectively.

By the end of Week 3, I had a solid understanding of the database structure and had implemented crucial optimizations to improve performance. Despite the challenges posed by the unstructured data, the research and exploratory efforts laid the groundwork for effective data analysis in the coming weeks.

11.4. Week 4

Week 4 was dedicated to structuring the data, re-evaluating the initial proposal, and finalizing project planning and task allocation.

Data Structuring: Utilizing Python within Jupyter Notebook, I connected to the MySQL client to further analyze the data, table structures, and relationships. This allowed me to employ Python for cleaning the data, effectively removing missing or garbage values. This data structuring process was crucial for preparing the data for subsequent analysis and ensuring its integrity.

Proposal Reevaluation: After gaining a foundational understanding of the database structure, I reevaluated the initial proposal and the metrics we planned to use. This reassessment involved checking the feasibility of the proposed metrics based on the actual database structure and devising new metrics where necessary. This step ensured that the proposal was both realistic and aligned with the available data.

Project Planning: We conducted a comprehensive team call to discuss all deliverables and create a detailed project roadmap. During this meeting, the insights computation and analysis were split into User and Organization sections, ensuring a focused approach. A plan was also developed for creating the web application, selecting the most suitable front-end and back-end technology stack. This included researching the best front-end technology and data visualization tools for the project.

By the end of Week 4, I had a well-structured dataset, an updated and feasible proposal, and a detailed project plan with clearly defined tasks and responsibilities. This set the stage for the development and implementation phases in the upcoming weeks.

11.5. Week 5

Week 5 focused on refining the proposed metrics, calculating and preparing data, and developing new features through comprehensive feature engineering.

Weekly Meeting with Mr. Rob Moore and Team Discussion: We had a productive meeting with Mr. Rob and the team where we reviewed the reevaluated metrics proposed by the team. I clarified any questions and addressed uncertainties regarding the database and its structure. Mr. Rob provided valuable feedback on the metrics list, and I discussed incorporating new ideas into our proposal to enhance our approach.

Metric Calculation and Data Preparation: I divided the calculation of different metrics and features for users and organizations among the three team members. This involved identifying and merging the necessary tables for analysis, utilizing specific columns to extract the required information. To ensure accurate calculations, I refreshed my knowledge of using Python libraries and statistical methods by watching tutorials online.

Feature Development and Engineering: I developed features such as the completion rate of user activities, wellness scores, peak active times, and more. This involved conducting feature engineering to create new tables and columns for comprehensive data analysis, ensuring that all necessary data was structured and ready for further insights.

By the end of Week 5, I had made significant progress in refining our metrics, preparing the data, and developing key features. This groundwork was essential for the continued success of the project, ensuring that I was on track with the goals and ready to tackle the next phases.

11.6. Week 6

Week 6 was dedicated to refining the system architecture, integrating code, setting up the final database, and ensuring effective communication between the system components.

System Architecture: I began the week by completing a refresher course on REST APIs using Flask, which provided the necessary foundation to develop an architecture diagram detailing the system setup. The diagram established the groundwork for the Flask app and outlined the planned communication flow between the frontend, backend, and database, ensuring seamless interaction among all components.

Weekly Meeting with Mr. Rob Moore: During our weekly meeting with Mr. Rob, I discussed the derived metrics and addressed concerns regarding data integrity. Mr. Rob's feedback was crucial in refining the approach to data handling and metric calculation, allowing me to make necessary adjustments to the analysis.

Database Listener Research: I also focused on researching and reviewing methods for setting up listeners on MySQL through Python. This involved exploring various approaches and best practices for implementing database listeners to monitor changes and trigger actions accordingly. This research was aimed at enhancing our system's responsiveness and data accuracy.

Code Integration and Database Setup: In addition, I designed the new ER diagram for the Wellnify app, creating a detailed database schema and documenting the relationships between tables. This design was crucial for structuring our final database effectively. Finally, I merged all the code from different notebooks created by various team members and pushed the new tables and features to establish our final database. This integration ensured that all data was properly organized and accessible for analysis, setting a solid foundation for the next phases of development.

By the end of Week 6, I had a refined system architecture, an updated project plan, and a fully integrated database, positioning me well for the continued success of the project.

11.7. Week 7

Week 7 focused on providing updates, addressing internal challenges, optimizing the staging database, and developing a proof of concept for trigger functionality.

Weekly Meeting with Mr. Rob Moore: During my weekly meeting with Mr. Rob, I provided updates on the project's progress, highlighting the new features added to the database. We explored the possibility of working with the staging database to achieve a better data structure and discussed potential improvements to enhance data management and integrity.

Internal Team Discussion: We conducted an internal team discussion to further plan the project. Each team member shared insights on individual problems faced with the data, and we brainstormed ideas to work around these issues. This collaborative effort led to the division of frontend and backend tasks among the team members, ensuring that responsibilities were clearly defined and manageable.

Access and Optimization of Staging Database: I gained access to the staging database and optimized the code in the python scripts to dynamically use the provided URL or configuration of the database host. The code now utilizes the existing data and tables in the database to calculate required features, creating new tables and columns, and then pushing these updates back to the database. This optimization minimized redundant code and reduced the initial setup required when transitioning between local, development, staging, and eventually production databases. Additionally, I, along with the team, transitioned from working with local databases to setting up a database on AWS to ensure synchronicity within the team. The final SQL database was hosted on AWS for consistent and scalable access, promoting efficient collaboration and data management.

Proof of Concept Development for Trigger Functionality: I developed a proof of concept to replicate the functionality of database triggers using Flask. This involved attempting to populate tables in real-time without using traditional triggers, by scheduling functions written in Jupyter Notebook. This approach aimed to streamline data processing and improve system responsiveness.

By the end of Week 7, we had made significant progress in optimizing our database setup, refining our project plan, and developing innovative solutions for data management and processing. These efforts positioned us well for continued success in the project's subsequent phases..

11.8. Week 8

Week 8 focused on setting up the Flask environment, documenting necessary APIs, creating a Git repository, and updating the backend structure based on team discussions.

Set Up the Initial Flask Environment: I started by installing Flask and its dependencies, ensuring that the development environment was properly configured. This setup was crucial for developing and running our web application smoothly. Next, I listed and planned the necessary API endpoints for the project. This involved establishing consistent naming conventions and

coding standards to ensure clarity and maintainability in our codebase. To manage our code effectively, I set up the directory layout and configuration files for a new Git repository. This repository would be the central location for all project-related code, facilitating version control and collaboration.

Updated the Backend Structure: Based on our team discussion, I updated the backend structure to align with our revised plan. These updates ensured that the backend was well-organized and capable of supporting the new features and requirements we had identified.

By the end of Week 8, I had established a solid Flask environment, documented the APIs, organized the code repository, and refined our backend structure.

11.9. Week 9

Week 9 focused on developing user-related APIs, enhancing features based on feedback, fixing bugs, and providing project updates in meetings.

User API Development: I was tasked with creating APIs for user-related metrics. During this week, I developed all the necessary APIs to capture and process user metrics. After development, I thoroughly tested the APIs to ensure their functionality and reliability, making sure they performed as expected under various conditions.

Feature Enhancement: Based on feedback from the frontend developer, I worked on adding more user features that could be displayed as charts, providing over-time insights. This involved conducting research and brainstorming sessions to explore additional features and metrics that could be beneficial for users. The aim was to enhance the user experience by offering more detailed and insightful data visualizations.

Bug Fixes and Changes: In response to feedback from the frontend developer, I performed several bug fixes and made necessary changes to the code. These adjustments were crucial for ensuring the smooth operation of the APIs and improving the overall user experience.

By the end of Week 9, significant progress had been made in developing user APIs, enhancing features, fixing bugs, and ensuring clear communication through regular meetings. This laid a strong foundation for the continued development and improvement of the project.

11.10. Week 10

Week 10 was dedicated to enhancing user engagement analysis, fixing bugs, updating APIs, and preparing for an important demo meeting.

Enhancing User Engagement Analysis: I added new features and tables for user engagement analysis and integrated these into the database. The notebook code was modified to conduct the initial analysis and store the new features in the tables. Additionally, the notebook code was optimized and cleaned for better performance and readability, ensuring efficient data processing and accurate analysis.

Bug Fixes and Changes: In response to feedback from the frontend developer, I performed several bug fixes and made necessary changes to the code. These adjustments were essential to ensure the smooth operation of the APIs and improve the overall user experience. Following the internal team meeting, I performed additional bug fixes and changes based on the identified issues. These final adjustments were critical to ensure a smooth and successful demo presentation.

User API Development: I added new APIs for the recently developed features and thoroughly tested their functionality. This ensured that the new APIs were reliable and effectively supported the enhanced features.

Internal Team Discussion: We held an internal team meeting to review the entire project and conduct thorough testing before the upcoming demo meeting with Mr. Rob and Mr. Brendon. This meeting allowed us to identify and address any issues, ensuring that the project was in the best possible shape for the demo.

By the end of Week 10, I had made significant strides in enhancing user engagement analysis, updating APIs, and preparing for an important demo meeting.

11.11. Week 11

Week 11 focused on a comprehensive demo meeting with stakeholders, enhancing and integrating code, and outlining future development steps.

Meeting with the Stakeholders: I held a meeting with Mr. Rob and Mr. Brendon to give a comprehensive demo of the entire app. During the demo, we discussed potential enhancements to the metrics and how they are calculated. We explored how the new features and metrics would be integrated into their existing system. Additionally, we outlined future steps for continued development and improvement of the app. This meeting was crucial for aligning my progress with stakeholder expectations and receiving valuable feedback for further refinement.

Code Enhancement and Integration: Throughout the week, I worked on enhancing and integrating the code. This involved restructuring the code in the Jupyter Notebook to enhance readability and adding logging to the scripts according to industry standards. I combined the notebook code and backend code into a single repository, encountering and resolving multiple errors in the process. Furthermore, I implemented logic in the backend to integrate and execute the notebook scripts, ensuring seamless functionality and improved performance.

By the end of Week 11, I had successfully demonstrated the app to the stakeholders, received valuable feedback for further enhancements, and made significant progress in enhancing and integrating the code. These efforts ensured that the project continued to move forward smoothly and was well-prepared for future development phases.

11.12. Week 12

In Week 12, the primary focus was on building and refining Docker configurations, troubleshooting triggers, and transitioning from Jupyter Notebooks to standalone Python scripts to streamline the development process.

Setting Up Docker Configurations: To ensure consistency across development environments, I began by creating Docker configurations for each Python script. This involved setting up individual Dockerfiles to manage dependencies and automate execution through scheduled cron jobs. Each container was carefully tested to verify it executed the script as expected, which was crucial for maintaining reliable data processing in the absence of traditional triggers.

Resolving Trigger Issues and Team Collaboration: During the week, I encountered and resolved specific issues with the database trigger functionality, testing thoroughly to verify the trigger's reliability. To address these challenges, I collaborated with Mr. Rob, where we discussed potential solutions and received feedback to guide further improvements. After internal discussions, the team decided to transition from Jupyter Notebooks to Python scripts to enhance execution speed and scalability.

Developing and Updating Python Scripts: Following the decision to shift away from Jupyter Notebooks, I developed standalone Python scripts that replicated and improved upon the data analysis functions originally implemented in the notebooks. Each script was validated for accuracy, and additional Docker configurations were created to support automated execution. By the end of the week, I had successfully set up these Python scripts within Docker containers, establishing a more efficient and consistent framework for the project's data processing tasks.

These efforts in Week 12 were essential to standardize the development environment, improve data processing efficiency, and address the challenges associated with database triggers, setting a solid foundation for continued project development.

11.13. Week 13

In Week 13, the focus was on conducting comprehensive testing, resolving final bugs, and preparing for the stakeholder wrap-up meeting to present project outcomes.

Comprehensive Testing: I conducted extensive testing of the dashboard interface, Python scripts, data analysis processes, and overall system functionality to ensure a seamless user experience.

This testing involved validating data accuracy, assessing script performance, and checking the responsiveness of the dashboard. Each component was thoroughly reviewed, and any issues encountered were documented for follow-up action, ensuring that the system was stable and reliable.

Internal Team Discussion: An internal team meeting was held to review the project's current status and discuss any final adjustments needed before stakeholder review. We confirmed that all deliverables were complete and outlined next steps for final documentation. This discussion helped ensure alignment across the team and set clear objectives for wrapping up the project.

Bug Fixes and Changes: During testing, a few bugs were identified, which were promptly addressed to enhance system stability and functionality. Follow-up tests were conducted after each fix to confirm that the issues were resolved and that performance improvements were achieved. This final round of bug fixes was critical to delivering a polished and reliable product.

Meeting with Stakeholders: I presented the final project outcomes to the stakeholders, highlighting key accomplishments and discussing the overall impact of the work. The meeting included a wrap-up discussion where we reflected on lessons learned and explored ideas for potential future projects. We also discussed ways to expand Wellnify's capabilities, laying the groundwork for continued collaboration and development in the wellness technology space.

These activities in Week 13 were essential for completing the project, ensuring quality, and setting a foundation for future growth.

11.14. Week 14

In Week 14, I on exploring and planning the next steps for Wellnify's recommendation system, with a specific emphasis on evaluating project ideas and selecting the best approach for future development.

Next Project Ideas Brainstorming: I participated in a collaborative brainstorming session with the team to discuss potential project directions that would align with Wellnify's mission and drive further user engagement. Each idea was evaluated based on feasibility, potential impact, and alignment with current wellness trends. We explored recent advancements in recommendation systems and documented key takeaways from our discussion, including initial insights on which projects could add the most value for Wellnify users.

Exploring Recommendation System Approaches: I conducted an in-depth analysis comparing Large Language Model (LLM)-based and traditional Machine Learning (ML)-based recommendation systems. This involved creating a comparison matrix that outlined the pros, cons, resource requirements, and estimated timelines for each approach. Additionally, I drafted a preliminary outline for how each method could integrate with Wellnify's existing data architecture. A follow-up discussion with the team helped refine our comparison and address questions, bringing us closer to a decision.

Research on LLM-Based and ML-Based Systems: I performed extensive research on best practices for both LLM-based and ML-based recommendation systems, with a focus on performance with large datasets and various data types. This research included examining tools and libraries such as the Transformers library for LLMs and Scikit-Learn and TensorFlow for ML-based systems. I also analyzed case studies from similar wellness and engagement platforms to understand practical implementation challenges. To guide our approach, I compiled a list of potential datasets and features for each method, considering availability and compatibility with Wellnify's platform.

Decision Meeting and Documentation: In a meeting with Mr. Rob, I presented our findings, emphasizing the strengths and weaknesses of both LLM-based and ML-based approaches. After discussing project goals, resources, and technical considerations, we collectively decided to proceed with an ML-based recommendation system for the initial implementation. Following the meeting, I documented our research findings, project scope, and technical requirements for the

ML-based approach. This documentation included an overview of model design, feature engineering ideas, and a visual roadmap outlining the project timeline and key phases. Week 14's efforts established a strong foundation for the next stage of Wellnify's recommendation system development.

11.15. Week 15

In Week 15, the team concentrated on structuring the project plan, refining the recommendation system's approach, and conducting in-depth research and analysis to lay a solid foundation for the development phase.

Developing the Project Plan: I began the week by creating a detailed project plan, which included specific tasks, deliverables, and timelines for each phase of the recommendation system's development. I assigned preliminary responsibilities to each team member, aligning their expertise with the project's needs, and outlined key milestones and checkpoints to track progress, including regular review sessions. Risk management strategies were also incorporated, identifying potential bottlenecks and outlining contingency plans. Additionally, I drafted initial testing and validation strategies to evaluate the model's effectiveness and prepare for potential iterations.

Evaluating Recommendation Approaches: To determine the optimal recommendation approach for Wellnify's needs, I compared content-based filtering with collaborative filtering, analyzing their strengths and weaknesses in terms of data availability, personalization, and scalability. After documenting how each approach aligned with Wellnify's objectives, I presented my findings to Mr. Rob. Based on the discussion, we agreed that a content-based recommendation model would be the most suitable choice, given its compatibility with Wellnify's current data structure and focus on user engagement. We then identified immediate tasks, such as feature extraction and model design, to kickstart the content-based system's development.

Production Database Review and Content-Based Research: I conducted a thorough review of the latest production database, examining user behavior data, content tags, and engagement metrics to identify features relevant to the recommendation model. I created a data map detailing key tables, relationships, and fields that would be essential for model training and scoring. To support this work, I explored tutorials, documentation, and case studies on content-based recommendation systems, focusing on feature engineering, similarity measures, and model evaluation techniques. I documented best practices and experimented with small-scale code snippets to test preliminary implementation approaches, ensuring they aligned with Wellnify's goals.

Internal Team Discussion: At the end of the week, I convened an internal team meeting to share findings from the database review and content-based research. We discussed initial steps for feature extraction and similarity metric selection to ensure alignment on the project direction. Upcoming tasks and responsibilities were outlined, with clear milestones set for the development phase. This meeting allowed us to address any initial questions or concerns related to data handling and model design, establishing a collaborative path forward for the content-based recommendation system.

11.16. Week 16

In Week 16, efforts were focused on setting up a robust data extraction, cleaning, and storage process for the recommendation model, involving coordination with stakeholders, internal team discussions, and hands-on data handling tasks.

Stakeholder Meeting and Team Discussion: At the beginning of the week, I met with the stakeholders to outline the data extraction requirements and discuss essential data elements for the recommendation model. This included identifying crucial tables and fields, such as collections, tag mappings, and activity links, which would form the foundation of our recommendation algorithm. Following this, an internal team discussion focused on data validation strategies, handling null values, and assigning specific tasks for structured data

extraction. We agreed on clear steps for validating extracted data to maintain consistency and accuracy throughout the process.

SQL Query Development for Data Extraction: With a clear understanding of data requirements, I created a series of SQL queries to extract key fields necessary for the recommendation model, ensuring that tag mappings, activity links, and supplementary fields like timestamps were included. Each query was tested on sample tables to validate accuracy, including manual checks of row counts and key field values. I documented the purpose and parameters of each query to ensure future reproducibility, implementing exception handling to manage any errors or missing data scenarios that might arise. This documentation and error-handling strategy helped build a reliable extraction process that would be resilient in the face of data inconsistencies.

Data Cleanup and Null Handling: After data extraction, I applied pandas-based functions to clean the data, focusing on handling missing values in critical fields like tags and activities. Conditional replacements were used to fill common gaps, and a process was established to identify and flag duplicate records across tables. Text fields, including tags and collection names, were standardized by converting them to lowercase and replacing spaces with underscores, preparing them for similarity calculations. To ensure data accuracy, I cross-referenced a subset of cleaned records with the original database entries, verifying that no essential information was lost. I documented each data cleaning step, capturing any edge cases handled to support future reproducibility.

Initial Data Storage Setup: To finalize the week's work, I organized the cleaned and validated data into structured CSV files, categorizing each dataset by its role within the recommendation pipeline. Metadata for each file, including row counts and data fields, was recorded to facilitate data lineage tracking. I conducted initial data profiling for each file, summarizing row counts, missing values, and unique entries in a report. This profiling provided insights into the dataset's structure and informed feature engineering strategies for subsequent stages of model

development. Week 16's work established a clean and well-documented data foundation, setting the stage for feature engineering and model training in the upcoming weeks.

11.17. Week 17

In Week 17, the focus was on preparing and transforming the dataset for feature engineering, encoding categorical data, and creating a standardized, consolidated feature matrix for the recommendation model.

Meeting with Mr. Rob: I began the week by presenting extracted data samples to Mr. Rob, showcasing the key fields and structure prepared for feature engineering. We discussed potential data transformations, including suitable encoding methods for categorical fields and strategies for handling hierarchical tags and activity categories. Mr. Rob provided valuable feedback on ensuring the dataset's scalability, especially as additional features may be integrated in the future. Based on this meeting, I created a follow-up action plan to address specific transformation needs that would help future-proof the dataset.

Internal Team Discussion and Encoding Strategy: Following the stakeholder meeting, the team convened to finalize encoding strategies for different types of features, such as tags, activities, and collection types. Recognizing that one-hot encoding could create a high-dimensional matrix for some categories, we explored alternative encoding methods, such as binary and frequency encoding, for features with a larger number of categories. A shared guide on encoding standards was drafted to maintain consistency, and responsibilities for encoding tasks were divided among team members, with each person focusing on a specific category of features.

One-Hot Encoding and Multi-Label Binarization: I applied one-hot encoding to categorical fields, such as tags, collection types, and activities, generating binary matrices for each. Rigorous testing was conducted to cross-check the encoded matrices against original data to ensure completeness. For features with multiple tag associations, I used `MultiLabelBinarizer` to encode multi-category tags into a binary format, capturing each possible combination of tags. Tests

verified the accuracy and compatibility of these encoded matrices with other data features, and I adjusted binarization settings to enhance performance.

Consolidation and Standardization of Encoded Features: After encoding, I consolidated all one-hot encoded and multi-label binarized features into a single, comprehensive feature matrix for model training. I removed any redundant columns resulting from overlapping categories and saved multiple versions of the dataset to enable experimentation with different feature combinations. To prepare the dataset for similarity calculations, I applied scaling and normalization techniques, such as Min-Max scaling for numerical fields and Z-score normalization for specific features. Lastly, I standardized all encoded features to ensure compatibility with similarity functions and documented each step of the transformation process, including a comprehensive report and visual diagrams to facilitate future reference. This week's efforts established a refined, high-quality feature matrix, setting a strong foundation for model development.

11.18. Week 18

In Week 18, the team focused on implementing text-based processing and vectorization to capture the semantic relevance of content for the recommendation model. This involved collaborative planning, applying TF-IDF vectorization, and optimizing the resulting matrices for efficient use in the recommendation system.

Stakeholder Meeting and Team Discussion: Early in the week, I met with stakeholders to discuss the application of TF-IDF for converting text-based content into numerical form, specifically to capture the relevance of terms in collection descriptions and tags. We reviewed alternative methods like word embeddings, ultimately determining that TF-IDF was suitable for our current data structure and scalability needs. This was followed by an internal team discussion to finalize workflows for text preprocessing, ensuring all relevant content fields were included. A workflow guide was drafted, detailing steps like tokenization, stop word removal, and lemmatization, with roles assigned to team members to maintain consistency across each stage.

Text Cleaning, Tokenization, and TF-IDF Vectorization: I began text processing by tokenizing each content summary, splitting it into individual words for further analysis. Non-informative characters and stop words were removed, and terms were standardized to enhance consistency across documents. After cleaning, I implemented TF-IDF vectorization using Scikit-Learn, transforming content summaries into numerical form based on term frequency and inverse document frequency. Experiments with various TF-IDF parameters (e.g., max_df, min_df, ngram_range) were conducted to optimize term importance. This process produced sparse matrices that captured the relevance of each word in context, and configurations were tested to find the one that best represented content for the recommendation system.

Dimensionality Reduction and Sparse Matrix Optimization: To improve memory efficiency, I applied Singular Value Decomposition (SVD) to the TF-IDF matrix, reducing its dimensionality without sacrificing core semantic information. Different component counts were tested, balancing dimensionality with representational accuracy. Once the optimal component count was determined, I consolidated the reduced TF-IDF matrix with previously encoded categorical features using SciPy's hstack function, creating a unified sparse matrix. I applied memory optimization techniques, selecting efficient data types to manage storage requirements, and verified alignment between matrix columns to ensure consistent representation across all features.

Testing, Validation, and Documentation: To ensure accuracy, I conducted validation tests on the sparse matrix, verifying that each vectorized feature matched the original data intent. Multiple test cases were run to confirm stable, accurate values across various collections, and unit tests were created for individual feature sets, capturing any edge cases. Comprehensive documentation was compiled, detailing each stage of text processing from tokenization to matrix consolidation. Additionally, I prepared a report on the optimization techniques and trade-offs in dimensionality reduction, providing a clear reference for future model development and

updates. Week 18's work established a well-optimized, text-based feature matrix ready for integration into the recommendation model.

11.19. Week 19

In Week 19, the primary focus was on developing and fine-tuning the similarity calculations for the recommendation model, specifically targeting cosine similarity and feature weighting to improve the personalization of recommendations.

Stakeholder Meeting and Team Discussion: At the start of the week, I presented potential similarity calculation techniques to stakeholders, emphasizing the use of cosine similarity and weighted feature scoring. We discussed approaches to prioritize certain attributes, like tags or activities, based on user engagement patterns, which would allow the model to provide more tailored recommendations. Mr. Rob provided feedback on which features should be prioritized, setting a foundation for iterative improvements. Following this, an internal team discussion reaffirmed the use of cosine similarity as the primary metric, while also considering alternatives like Euclidean and Jaccard similarity. We agreed to take a modular approach to similarity calculation, making it easier to adjust or add features in the future.

Implementation of Cosine Similarity and Modular Functions: I implemented cosine similarity for individual features—tags, collection types, and activities—using Scikit-Learn's `cosine_similarity` function. Parameters were fine-tuned based on preliminary test results to ensure that similarity scores accurately captured user content preferences. Each feature's similarity calculation was built into a modular function, allowing for easy maintenance and adjustments as needed. To ensure accuracy, I verified that feature-level similarities aligned with raw data, confirming that scores reflected expected relationships based on user interactions and content attributes.

Development of Weighted Feature Combination: After calculating feature-specific similarities, I developed a system to combine these scores, allowing adjustable weights for each feature type. This system enabled testing of various weight distributions to find the optimal balance between

feature influence, making it scalable for future changes in recommendation priorities. Multiple configurations were tested, with detailed documentation on each distribution's rationale, ensuring a comprehensive understanding of the chosen weight setup. This weighted approach helped enhance the model's adaptability, allowing us to emphasize features based on ongoing insights or shifting user engagement patterns.

Testing, Validation, and Documentation: Extensive testing was conducted on the similarity matrices across different user profiles to evaluate how well the model captured relevant recommendations. This testing identified potential biases, particularly in high-frequency tags, which were adjusted by refining weights and tuning scoring parameters. I compiled results from these test runs, documenting the most effective configurations for future reference. Finally, I combined the individual feature-specific matrices into a single unified similarity matrix, validating it through baseline comparisons and automated consistency checks. Documentation was created to outline the entire similarity calculation process, including tuning details, test results, and visual aids to illustrate the impact of feature weighting. This report was reviewed by the team, setting a solid foundation for future adjustments and refinements to the recommendation system.

11.20. Week 20

In Week 20, the primary objective was to fine-tune the weights assigned to different features within the recommendation model, using iterative testing and advanced weighting techniques to optimize the relevance and personalization of recommendations.

Stakeholder Meeting and Brainstorming: At the beginning of the week, I met with Mr. Rob to discuss initial similarity scores and the impact of feature weights on recommendation relevance. We reviewed user engagement data and feedback, identifying patterns that could guide weight adjustments, such as balancing tag, activity, and content-based similarities. Mr. Rob provided valuable insights on dynamically adjusting weights based on user behavior, and we agreed on a strategy to balance these features more effectively. Following this meeting, the team conducted a brainstorming session to explore techniques for weight adjustment, such as normalization,

incremental adjustments, and dynamic scaling. We created a reference guide on these techniques, identifying potential issues, such as the risk of over-emphasizing frequently occurring tags, and documented solutions to maintain balance in recommendations.

Iterative Weight Adjustment and Testing: I implemented an iterative tuning process, adjusting weights for each feature—TF-IDF, tags, and activities—across multiple test scenarios. A grid search method was used to systematically evaluate various weight combinations, and each test's impact on recommendation quality was recorded in detail. This iterative approach enabled us to identify configurations that yielded the highest engagement and relevance. Additionally, I documented insights from these tests, including optimal configurations and adjustments that improved recommendation effectiveness.

Applying Smoothed Inverse Frequency for Tag Weighting: To address the dominance of high-frequency tags, I applied a smoothed inverse frequency method, which re-weighted tags to reduce the influence of common tags while emphasizing unique or niche tags. I experimented with different smoothing factors to strike a balance, ensuring that rare tags contributed more to personalized recommendations. These adjustments were validated across various user profiles, confirming that inverse frequency weighting led to greater personalization and diversity in the recommendations. This method was documented thoroughly, and multiple configurations were saved for future reference.

User-Centric and Adaptive Weight Tuning: Building on our weight adjustments, I conducted additional tests with configurations tailored to specific user profiles, focusing on personalized recommendations based on individual user interaction data. Adaptive weighting methods were implemented to adjust based on user engagement patterns, providing recommendations that aligned closely with users' unique preferences. Findings from these profile-based tests were recorded, offering insights into refining weight tuning for user-specific recommendations.

Testing, Validation, and Documentation: Extensive validation tests were conducted to confirm the stability and accuracy of the final weight configurations across different data sets. Automated test cases were created to ensure the consistency of weight adjustments with new data, supporting ongoing model updates. A detailed report was compiled, documenting each step of the weight tuning process, the rationale behind selected configurations, and the final weight distribution. This report included a reference guide with recommended configurations, enabling easy replication and customization for future adjustments. Week 20's work established a robust, user-centric weighting framework, enhancing the adaptability and relevance of Wellnify's recommendation model.

11.21. Week 21

In Week 21, the team focused on designing, implementing, and testing API endpoints for the recommendation system, ensuring seamless delivery of recommendations to users while maintaining robust security and performance.

Stakeholder Meeting and Team Discussion: At the beginning of the week, I met with Mr. Rob to review the desired API structure, focusing on key endpoints necessary for real-time recommendation delivery. We discussed scalability requirements to handle high user loads and frequent requests, exploring caching strategies to minimize response times. Mr. Rob emphasized the importance of user privacy, and we gathered feedback on data security measures to safeguard sensitive information. Following this, the team held an internal discussion to finalize API endpoint specifications, security protocols, and error-handling strategies. We outlined the requirements for each endpoint, including HTTP methods, parameters, and data formats, to ensure a smooth integration process. A security plan was developed, incorporating token-based authentication and rate limiting to prevent abuse, and potential errors were documented with clear messages and status codes.

Implementation of Recommendation API Endpoints: I proceeded to develop multiple Flask-based endpoints for delivering recommendations, including options to retrieve

recommendations by user ID and filter by category or tags. Query parameters were added to allow customization, such as specifying the number of recommendations or filtering by content type, providing flexibility for different use cases. To optimize response times, caching mechanisms were implemented for frequently requested recommendations, and initial tests were conducted on each endpoint to ensure smooth functionality, handling both successful responses and error cases effectively.

Error Handling, Logging, and Integration Testing: Comprehensive error-handling logic was implemented to manage scenarios like invalid requests, data retrieval issues, and server errors. Logging was added at critical points within each endpoint to capture request details, response times, and any encountered errors for troubleshooting. I then conducted integration tests to validate the interaction between the API, recommendation model, and data pipeline, simulating end-to-end API requests to confirm accuracy, consistency, and speed. Load-testing tools were used to simulate high traffic, measuring response times and identifying potential bottlenecks. Based on these results, configurations were adjusted to enhance real-time responsiveness, ensuring stable performance under peak loads.

Security Testing, Authorization Configuration, and Documentation: Token-based authentication was configured to secure API endpoints, restricting access to authorized users. Security tests were performed to verify the protection of sensitive data, simulating potential attack vectors to test the API's robustness. Additional security measures, including encryption for data in transit and rate limiting, were implemented to prevent abuse. To conclude the week, I developed comprehensive documentation for each API endpoint, detailing input parameters, response structures, and expected error messages. A user-friendly guide was created with examples demonstrating common usage scenarios, along with best practices for secure API access. This documentation was reviewed with the team for clarity, setting the stage for a secure, reliable, and user-friendly recommendation API.

11.22. Week 22

In Week 22, the focus was on implementing and fine-tuning filtering criteria for dynamic recommendations within the API, along with establishing a logging system to track user interactions and analyzing engagement data.

Meeting with Mr. Rob and Discussion on Filter Criteria: Early in the week, I met with Mr. Rob to discuss requirements for dynamically retrieving recommendations in both organization-specific and user-specific contexts. We reviewed use cases for personalizing recommendations based on organizational filters to enhance engagement within user groups, balancing personalized results with response times. Following this, the team held a brainstorming session to finalize filtering criteria based on user preferences, engagement history, and organizational affiliation. We determined that filters such as age, activity level, and preferred content types would help align recommendations with user-specific interests while maintaining relevance across organizational themes.

Development of Organization-Based and User-Specific Filtering: I integrated organization-based filters into the API, allowing recommendations to be tailored to a user's organizational context by checking affiliations and filtering content based on the organization's preferences. Tests were conducted with sample data from various organizations, verifying that the organization-based filters aligned with specific organizational themes and dynamically adjusted as data changed. Additionally, I developed user-specific filtering, incorporating preferences based on past interactions, activity types, and engagement levels. This system was designed to update recommendations in real-time based on recent user interactions, ensuring recommendations reflected the latest user behavior. Extensive tests confirmed that user-specific filters accurately personalized content for individual users.

Implementation of Recommendation Logging for Interaction Tracking: To support data-driven improvements, I established a comprehensive logging system to capture user interactions with recommendations, tracking key metrics like click-through rates, time spent on content, and user

feedback. Logs were configured to categorize interactions by type, making it easier to analyze engagement trends across different recommendation categories. Log rotation was implemented to manage storage capacity, ensuring that logs were retained for analysis without overloading storage. This logging system verified that all essential interactions were captured, enabling a robust foundation for tracking and improving recommendation performance.

Initial Data Analysis and Documentation: A preliminary analysis of the logged interaction data provided insights into user engagement patterns, highlighting high-interest content categories and frequent user preferences. I used statistical methods to measure engagement trends, such as average interaction time and click-through rates, and compiled a report summarizing initial insights, noting popular recommendation filters and any observed interaction discrepancies. Documentation was prepared on the filtering logic for both organization-based and user-specific recommendations, along with a comprehensive guide on the logging system setup and usage. This documentation was reviewed with the team to ensure clarity and served as a reference for future tuning and enhancement of the recommendation model.

11.23. Week 23

In Week 23, the focus was on analyzing user interaction data, refining the recommendation model based on user feedback and A/B testing, and setting up a continuous improvement loop to ensure ongoing model optimization.

Stakeholder Meeting and Internal Team Review: At the start of the week, I presented an overview of the interaction logging data to Mr. Rob, highlighting key metrics such as engagement rates and time spent on recommended content. Initial insights into user behavior patterns were discussed, and feedback was gathered on specific areas of the model that could benefit from refinement. Following this, the team conducted a comprehensive review of the logging process to identify any inconsistencies or gaps in interaction tracking. We aligned on adjustments to improve data consistency and established a protocol for verifying logging accuracy, incorporating regular checks to maintain data quality.

User Feedback Analysis and Model Adjustment: I conducted an in-depth analysis of user feedback data to identify high-engagement content types, examining which recommendations resonated most with users. Patterns related to specific tags, content types, and activity levels were identified, providing insights into user preferences. Based on this analysis, I adjusted the model to prioritize high-engagement content, refining the recommendation algorithm to better align with observed user interests. Each adjustment was documented, capturing insights from the analysis for future model refinement sessions.

A/B Testing Setup and Iterative Model Refinement: To further evaluate model configurations, I designed and implemented A/B testing, comparing multiple setups to assess engagement across different configurations. Test groups were created to focus on variations in feature weights, similarity metrics, and content prioritization. Data logging was configured to capture metrics on engagement, retention, and user satisfaction for each configuration, establishing a basis for comparative analysis. Initial tests were conducted to confirm accurate tracking of results, ensuring that each configuration delivered distinct recommendation outcomes. Based on the A/B testing results, I refined model configurations to maximize engagement, focusing on metrics like click-through rates and session duration. Follow-up tests validated the improvements, and a report was compiled summarizing the findings and rationale for final adjustments.

Implementation of a Continuous Improvement Loop: To ensure ongoing model refinement, I established a continuous improvement loop that integrated user feedback and engagement data into the model tuning process. Procedures were set up for regular reviews of interaction data, enabling the model to evolve with changing user preferences. Certain aspects of the feedback loop were automated, including the collection and analysis of engagement metrics on a scheduled basis, to streamline the improvement process. Comprehensive documentation was created, detailing the continuous improvement framework, including tracking performance, reviewing insights, and applying updates. Visual representations of engagement data, A/B testing

results, and model adjustment impacts were added to enhance clarity, and feedback from the team was incorporated to ensure alignment with future goals.

11.24. Week 24

In Week 24, efforts were concentrated on preparing for the final demo, refining the user interface, optimizing performance, and making last-minute adjustments to the recommendation model for enhanced relevance.

Stakeholder Meeting for Demo Planning: At the beginning of the week, I met with stakeholders to review the structure of the upcoming demo and outline essential metrics to highlight, such as engagement rates and recommendation relevance. We discussed specific scenarios and features that would best showcase the model's impact, ensuring alignment with stakeholder interests. Mr. Rob provided feedback on key metrics and features to emphasize, which helped finalize the content and focus areas for a clear and impactful demonstration.

Team Demo Rehearsal and Final Adjustments: A full rehearsal of the demo was conducted with the team to gather feedback on the flow, user experience, and overall presentation. Based on this feedback, adjustments were made to improve the clarity and impact of visuals, transitions, and explanations. These refinements helped create a polished presentation that conveyed the model's functionality and value effectively.

User Interface Enhancements and Performance Optimization: To prepare the UI for demo readiness, I updated the interface with tooltips and brief descriptions to guide users through the recommendation features. Compatibility and responsiveness tests were performed to ensure a seamless presentation experience. In addition, I fine-tuned backend processes to support real-time interactions, focusing on response time and efficiency. Stress tests were conducted to confirm system stability under simulated high-traffic conditions, ensuring a smooth performance during the demo.

Final Model Tuning and Testing: I made final adjustments to feature weights and personalization settings based on recent feedback to enhance recommendation relevance. The model's performance was tested across various user profiles, verifying consistent quality and relevance in the recommendations. This final tuning phase ensured the recommendation system was fully optimized and ready for the demo, highlighting its effectiveness in delivering personalized and impactful recommendations.

11.25. Week 25

In Week 25, the team focused on finalizing the project for handoff, completing documentation, and ensuring a smooth transition for future maintenance and development.

Final Meeting with Mr. Rob and Mr. Brendon: A comprehensive wrap-up meeting was held with Mr. Rob and Mr. Brendon to review the project's final outcomes. I presented key performance metrics, including system stability and recommendation relevance, and highlighted areas for potential future development. Valuable feedback was gathered, with insights on ongoing model refinement and suggested improvements for future iterations. We outlined next steps for system maintenance, ensuring alignment on goals for continuous monitoring and potential expansions, setting a clear direction for the platform's future growth.

Internal Team Review of Documentation and Integration: The team conducted a final review of all project documentation, ensuring completeness and accuracy for future reference. We verified integration points to facilitate a seamless handoff process and made sure that onboarding materials were readily available to support new team members. Any last-minute questions from team members about system components were addressed, establishing a smooth transition plan for continued maintenance.

Comprehensive Documentation of System Architecture and Codebase: I compiled extensive documentation for the entire codebase, covering API specifications, data pipelines, recommendation model setup, and deployment configurations. Clear diagrams and flowcharts

were created to illustrate the system's architecture, data flow, and integration points, ensuring ease of use for future developers. The documentation was organized into structured sections for easy navigation, tailored to various audiences, including developers and stakeholders, to provide a comprehensive reference.

Final Quality Assurance and Testing: A final quality assurance check was performed across the system to ensure stability and functionality. This included testing API response times, recommendation accuracy, and logging, under various scenarios to confirm system resilience. A summary report of the quality assurance results was compiled, including minor adjustments made and recommendations for periodic testing routines and maintenance tasks to uphold system performance.

Project Handoff Documentation and Report: I prepared a concise project handoff document that summarized key project components, outcomes, and areas for future development. This document included links to detailed documentation, resources, and team contacts for easy access by future developers. The report was reviewed with the team, confirming it provided a complete and clear overview of the system's current state, thus facilitating a seamless handoff and supporting future efforts to maintain and expand the platform.

12. CONCLUSIONS

In conclusion, I have contributed significantly to two pivotal projects—the User Engagement Dashboards and the Recommendation System for User Engagement—that have markedly enhanced the platform's capabilities in data analysis, visualization, and personalized engagement. The integration of sophisticated analytical tools and machine learning technologies not only advanced the platform's functionality but also aligned closely with Wellnify.ai's mission to foster healthier communities. These projects demonstrated the power of data-driven insights in shaping health behaviors and emphasized the importance of adaptive, user-centered design in technology development. As Wellnify.ai continues to evolve, the groundwork laid by these

initiatives promises further innovations. Continued refinement of the technologies based on user feedback and emerging trends will ensure the platform remains at the forefront of wellness technology. The experience has not only propelled Wellnify.ai towards becoming a leader in its field but has also significantly enriched my professional expertise, underscoring the dynamic interplay between technology and wellness in driving societal health improvements..

12.1. Learning Goals Overview

S.N.	Task	Description	Status
1	Master Python Programming	Develop extensive proficiency in Python, focusing on advanced features and best practices. Gain in-depth knowledge of key Python libraries such as NumPy, Pandas, and Matplotlib for data analysis, as well as Flask for web development.	Completed
2	Understand Data Analysis Techniques	Learn thorough data cleaning, pre-processing, and analysis techniques to extract meaningful insights from large datasets.	Completed
3	Develop Skills in REST API Development	Acquire hands-on experience in creating and managing REST APIs using Flask to support the web application's backend.	Completed
4	Explore Database Management	Understand the MySQL database management, including data migration and integration with web applications.	Completed
5	Build Proficiency in Machine Learning	Gain hands-on experience in developing machine learning models, specifically recommendation algorithms. Focus on implementing similarity measures such as TF-IDF vectorization and cosine similarity to deliver personalized recommendations.	Completed

6	Software Development Lifecycle (SDLC)	Gain a comprehensive understanding of the SDLC, including phases such as planning, analysis, design, implementation, testing, deployment, and ongoing maintenance. Learn to apply these principles in developing robust and efficient software solutions.	Completed
7	Understand the Wellness Technology Industry	Gain a comprehensive understanding of the wellness technology sector, focusing on how technology can enhance user engagement and improve health outcomes.	Completed
8	Learn About User Engagement Strategies	Study various strategies to boost user engagement through data-driven insights and personalized content.	Completed
9	Enhance Communication Skills	Develop effective communication skills to collaborate with team members and stakeholders, ensuring clear and concise information exchange.	Completed
10	Understand Business Processes	Learn about the business processes at Wellnify.ai, including project management.	Completed

Table 1: Learning Goals Overview

12.2. Timesheet

My Internship term at Wellnify began on the 14th of May, 2024 and ended on the 29th of October, 2024.

Work	Duration (in hours)
Wellnify work-term on report	616 hrs
Report Writing	120 hrs

Table 2: Timesheet

13. ACHIEVEMENTS

Working on the Wellnify project, I achieved several significant milestones across various domains, from data analysis to web development. These accomplishments highlight my growth and the skills gained during the project:

- **Python and Data Engineering:** Enhanced my proficiency in Python and successfully performed complex data engineering tasks, including data extraction, cleaning, and transformation for both the recommendation system and dashboard metrics.
- **User-Centric Web and API Development:** Improved my ability to design and develop web platforms with a focus on user accessibility and non-functional requirements. Gained hands-on experience in creating and managing REST APIs using Flask.
- **Backend Optimization:** Optimized backend processes for better performance and scalability.
- **Database Management:** Acquired experience in MySQL database management, including data migration, schema design, and integration, as well as creating indexing strategies to enhance data retrieval speed for frequently accessed metrics.
- **Machine Learning and Model Development:** Developed and fine-tuned a recommendation engine using machine learning techniques, implementing TF-IDF vectorization and cosine similarity calculations. Gained hands-on experience in iterative model tuning and feature weighting for improved recommendation accuracy.
- **Dashboard Processing and Feature Engineering:** Engineered multiple dashboard metrics, including custom engagement scores, peak activity indicators, and consistency metrics, providing stakeholders with a comprehensive view of user behavior and content performance.
- **Software Development Lifecycle (SDLC):** Applied SDLC principles to create robust and efficient software solutions, including thorough testing, iterative improvement, and documentation practices to ensure reliable system deployment.

- **User Engagement and Personalization Strategies:** Designed and implemented strategies to boost user engagement through data-driven insights and personalized content, utilizing behavioral segmentation, activity-based recommendations, and targeted content displays.
- **Data Pipeline and Automation:** Built a scalable data pipeline for the automated calculation of engagement scores, activity summaries, and recommendation results, supporting timely and consistent data updates for the dashboard and recommendation system.
- **Analytical Skills and Statistical Techniques:** Applied advanced analytical and statistical techniques, including feature scaling, smoothed inverse frequency weighting, and behavioral analysis, to refine data insights and enhance model performance.
- **Wellness Technology Industry Insight:** Gained a comprehensive understanding of the wellness technology sector and its impact on user engagement and health outcomes, learning how data and personalized recommendations can drive healthier behavior and increase user motivation.

These achievements reflect my ability to tackle complex challenges and contribute significantly to the success of the Wellnify.ai platform.

14. CHALLENGES

Throughout the Wellnify project, I encountered numerous complex challenges, particularly in managing data quality and navigating intricate database structures. The development database contained multiple tables with unstructured data and outdated records, which made data extraction and analysis difficult. Cleaning and preprocessing this data required meticulous, time-intensive work to ensure integrity across all records. Custom data-cleaning functions were essential to standardize the data, allowing for the creation of reliable metrics. This experience reinforced the importance of data quality and honed my skills in data engineering.

The dashboard development posed its own complexities, particularly in calculating engagement metrics and managing data aggregation at various levels (daily, weekly, monthly). Handling these calculations efficiently required designing a series of optimized aggregation functions to support real-time updates, allowing the dashboard to display accurate, up-to-date data. Balancing frontend and backend tasks while maintaining synchronization was also crucial. This process, driven by iterative feedback from stakeholders and frontend developers, required adaptability, clear communication, and agile coordination to keep both the frontend and backend in sync.

Integrating the recommendation model with the backend introduced additional challenges, especially in balancing real-time processing demands with system responsiveness. The model's development required optimizing similarity calculations to deliver accurate recommendations without overloading the system. This involved extensive tuning of parameters and managing memory to achieve computational efficiency. Additionally, because traditional database triggers were unavailable, I had to implement custom scheduling functions to refresh data consistently. This solution ensured real-time performance and deepened my understanding of backend optimization and system architecture constraints.

Finally, developing a user-centered recommendation engine involved significant feature engineering and iterative tuning. Incorporating diverse features like TF-IDF scores, activity data, and user-specific behavior required extensive testing to balance relevance and performance. Implementing custom weighting for these features, while also managing performance scalability, proved demanding. Through careful optimization and continuous feedback from users, I refined the recommendation engine and dashboard to align with project goals. These challenges strengthened my skills in data processing, system scalability, and user-centered design, and were essential to delivering a high-performance, engaging platform.

15. FUTURE WORK

15.1. User Engagement Dashboards

Looking ahead, there are several areas for further development and enhancement of dashboards. One key focus will be on refining the user engagement metrics and incorporating advanced analytics to provide deeper insights. This will involve leveraging machine learning techniques to predict user behavior and tailor personalized recommendations, thereby enhancing the overall user experience. Additionally, we plan to expand the data visualization capabilities of the platform, integrating more sophisticated tools and interactive elements to make the insights more accessible and actionable for users.

Another critical area for future work is the continuous improvement of the backend infrastructure. This includes optimizing database performance, scaling the backend systems to handle a growing user base, and ensuring robust security measures are in place to protect user data. Regular updates and iterations based on user feedback and technological advancements will be crucial to maintaining the platform's relevance and effectiveness in the dynamic wellness technology landscape.

15.2. Recommendation System for User Engagement

Further development of the recommendation model will focus on enhancing personalization and prediction accuracy by incorporating advanced machine learning techniques, such as collaborative filtering and user behavior modeling. These approaches will allow the model to better predict user preferences based on historical data and dynamically adapt to new activity patterns. Additionally, implementing a feedback loop to capture real-time user engagement data can enable continuous improvement, allowing recommendations to evolve in response to changing user behavior and interests.

Another key area for future work is optimizing the model's scalability and performance to support a growing user base. This will involve streamlining the backend to handle larger datasets and considering cloud-based or distributed computing solutions for efficient, real-time processing.

Expanding the feature set used in recommendations—by integrating additional user data and external wellness-related information—can further refine recommendations to be more holistic and aligned with individual wellness goals. These enhancements, along with ongoing data privacy safeguards, will ensure the recommendation system remains robust, responsive, and highly relevant as the platform evolves.

16. APPENDIX

16.1. References

Wellnify AI

- **Website:** <https://wellnify.ai/>
- **Description:** A platform dedicated to enhancing wellness through AI-driven insights and interventions.

Python

- **Website:** <https://www.python.org/>
- **Description:** Python is a versatile and widely-used programming language known for its readability and broad range of applications from web development to data analysis.

Jupyter

- **Website:** <https://jupyter.org/>
- **Description:** Jupyter offers powerful tools for interactive data science and scientific computing that allow users to create and share documents that contain live code, equations, visualizations, and narrative text.

Pandas

- **Website:** <https://pandas.pydata.org/>
- **Description:** Pandas is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation tool, built on top of the Python programming language.

NumPy

- **Website:** <https://numpy.org/>
- **Description:** NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays.

ChatGPT

- **Usage:** Used for paraphrasing purposes

- **Description:** ChatGPT is an AI developed by OpenAI that excels in understanding and generating natural language or text based on the input it is given.